



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Departamento de Informática de Sistemas y Computadores
Universitat Politècnica de València

Integración de Datos IoT, Modelos de Forecasting y Clasificación en Smartphones para Sistemas Edge y Fog

TRABAJO FIN DE MÁSTER

Máster Universitario en Ingeniería de Computadores y Redes

Autor: Gatti Ceaglio, Tobías

Tutor: Manzoni, Pietro
Blanc Clavero, Sara

Curso 2025-2026

Agradecimientos

Pidan y se les dará, busquen y encontrarán, toquen a la puerta y la puerta se abrirá para ustedes. Todo el que pide, recibe; todo el que busca, encuentra; a todo el que toca, se le abre la puerta. Acaso alguno de ustedes le daría una piedra a su hijo si este le pide un pan? ¿O si le pidiera un pez, le daría una serpiente? De modo que si incluso ustedes que son malos saben dar cosas buenas a sus hijos, cuánto más el Padre celestial dará cosas buenas a quienes le piden. **Traten a los demás como quieren que los traten a ustedes.**

- Pietro Manzoni y Sara Blanc.
- Compañeros de trabajo: Cesar [4] y Antonio [8].
- Novia.
- Familia, cercana y lejana.
- Amigos.
- Profesores UPV.

Gracias.

Resum

Aquest Trabajo de Fin de Máster (TFM) aborda l'optimització del reg en l'agricultura de precisió mitjançant la integració de xarxes de sensors IoT i models predictius executats directament en *smartphones*, consolidant una arquitectura descentralitzada d'*Edge* i *Fog Computing*. Davant l'escassetat hídrica global i les deficiències dels sistemes dependents del núvol, com ara l'alta latència, l'elevat consum energètic i la pèrdua de privacitat, aquest projecte proposa un ecosistema de baix cost i alta resiliència. Les dades microclimàtiques recollides per nodes IoT (*Deep-Edge*) es transmeten via Bluetooth a una aplicació mòbil multiplataforma que executa localment xarxes neuronals i arbres de decisió dinàmics. Així mateix, l'arquitectura integra un servidor *Fog* autoallotjable per a la sincronització transaccional i l'actualització en calent dels models estadístics. L'avaluació del sistema demostra un consum mínim de recursos computacionals en el dispositiu mòbil i projecta un estalvi hídric significatiu mitjançant simulacions empíriques. L'objectiu final és demostrar la viabilitat de democratitzar la Intel·ligència Artificial en entorns rurals, reduint la latència, garantint la sobirania de les dades de l'agricultor i promovent la sostenibilitat mediambiental.

Paraules clau: IoT; Edge AI; programació de *smartphones*

Resumen

El presente Trabajo de Fin de Máster (TFM) aborda la optimización del riego en la agricultura de precisión mediante la integración de redes de sensores IoT y modelos predictivos ejecutados directamente en *smartphones*, consolidando una arquitectura descentralizada de *Edge* y *Fog Computing*. Ante la escasez hídrica global y las deficiencias de los sistemas dependientes de la nube, como su alta latencia, elevado consumo energético y pérdida de privacidad, este proyecto propone un ecosistema de bajo coste y alta resiliencia. Los datos microclimáticos recolectados por nodos IoT (*Deep-Edge*) se transmiten vía Bluetooth a una aplicación móvil multiplataforma que ejecuta localmente redes neuronales y árboles de decisión dinámicos. Asimismo, la arquitectura integra un servidor *Fog* auto-alojable para la sincronización transaccional y la actualización en caliente de los modelos estadísticos. La evaluación del sistema demuestra un consumo mínimo de recursos computacionales en el dispositivo móvil y proyecta un ahorro hídrico significativo mediante simulaciones empíricas. El objetivo final es demostrar la viabilidad de democratizar la Inteligencia Artificial en entornos rurales, reduciendo la latencia, garantizando la soberanía de los datos del agricultor y promoviendo la sostenibilidad medioambiental.

Palabras clave: IoT; Edge AI; programación de *smartphones*

Abstract

This master's thesis (TFM) addresses the optimization of irrigation in precision agriculture by integrating IoT sensor networks with predictive models executed directly on smartphones, establishing a decentralized Edge and Fog Computing architecture. In the face of global water scarcity and the limitations of cloud-dependent systems—such as high latency, significant energy consumption, and privacy concerns—this project proposes a low-cost, highly resilient ecosystem. Microclimatic data collected by IoT nodes (*Deep-Edge*) are transmitted via Bluetooth to a cross-platform mobile application that locally executes neural networks and dynamic decision trees. Furthermore, the architecture incorporates a self-hostable Fog server for transactional synchronization and hot-reloading of statistical models. System evaluation demonstrates minimal computational

resource consumption on the mobile device and projects substantial water savings through empirical simulations. The ultimate goal is to prove the feasibility of democratizing Artificial Intelligence in rural settings, reducing latency, ensuring farmers' data sovereignty, and fostering environmental sustainability.

Key words: IoT; edge AI; smartphone programming

Índice general

Índice general	VII
Índice de figuras	IX
Índice de tablas	X
1 Introducción	1
1.1 Motivación	1
1.2 Problemática	1
1.3 Objetivos	2
1.3.1 Objetivo General	2
1.3.2 Objetivos Específicos	2
1.4 Propuesta y Filosofía Arquitectónica	3
1.5 Estructura de la Memoria	3
2 Análisis y Fundamentos	5
2.1 Agricultura de Precisión	5
2.1.1 Redes de Sensores Inalámbricos de Bajo Costo	6
2.1.2 Calidad y Fiabilidad del Dato	7
2.2 <i>Edge Computing</i> y <i>Cloud Computing</i>	8
2.3 <i>Machine Learning</i>	9
2.4 Arquitecturas Móviles para <i>Edge Computing</i>	10
3 Arquitectura Global del Sistema	13
3.1 Logística	15
3.2 Flujo de Datos	16
3.3 Seguridad y Privacidad de Datos	17
3.3.1 Superficie de Ataque y Mitigación en el Enlace Inalámbrico	18
3.3.2 Protección en la Capa Móvil	18
3.3.3 Seguridad Perimetral en el Servidor <i>Fog</i> Auto-Alojable	18
3.3.4 Soberanía y Privacidad de los Datos Agrícolas	19
4 Desarrollo de la Aplicación Móvil	21
4.1 Módulo Central	22
4.1.1 Base de Datos	22
4.1.2 Modelo de Datos	24
4.1.3 Red	25
4.1.4 Utilidades y Diseño	26
4.2 Módulo de Características	26
4.2.1 <i>Bluetooth</i>	26
4.2.2 <i>Machine Learning</i>	27
4.3 Módulo de Interfaz de Usuario y Gestión de Estado	27
4.3.1 Menú Principal o <i>Dashboard</i>	28
4.3.2 Menú de Conexión <i>Bluetooth</i>	29
4.3.3 Menú de Almacenamiento y Persistencia	30
4.3.4 Menú de Configuración Paramétrica	31
5 Integración de <i>Hardware</i> y Flujo de Telemetría	35

5.1	Codesarrollo de Protocolos y <i>Firmware</i>	35
5.2	Protocolo de comunicación Bluetooth Low Energy (BLE)	37
5.2.1	Perfil de Atributos Genérico GATT	38
5.2.2	Estructuración del <i>Payload</i>	39
5.2.3	Estabilidad de Conexión y Manejo de Errores	40
6	Almacenamiento Centralizado y Servidor Auto-Alojable	43
6.1	Arquitectura del servidor y <i>stack</i> tecnológico	43
6.2	Esquema de base de datos	44
6.3	Diseño de la API RESTful	46
6.4	Estrategia de sincronización bidireccional	47
6.5	Catalogación y Conversión de Modelos Predictivos	47
6.5.1	Algoritmo de Extracción e Introspección	48
7	Orquestación e Integración del <i>Machine Learning</i> (ML)	51
7.1	Orquestación y Flujo de Datos	51
7.2	Fusión de Telemetría <i>IoT</i> y Meteorología	51
7.3	Adaptación de Modelos al <i>Smartphone</i>	52
7.4	El Flujo de Datos del Veredicto Final	53
8	Evaluación y Validación del Sistema	55
8.1	Despliegue Multiplataforma y Ecosistema de Red	55
8.2	Validación Agronómica mediante <i>Mocking</i> de Telemetría	55
8.2.1	Perfilado de Recursos y Escenarios de Prueba	56
9	Conclusiones y Trabajo Futuro	57
9.1	Cumplimiento de los objetivos	57
9.2	Principales contribuciones técnicas	58
9.3	Impacto ambiental y operativo	58
9.4	Líneas de investigación futuras	58
	Bibliografía	61

Índice de figuras

2.1	Diagrama conceptual de la arquitectura de co-diseño híbrido.	9
3.1	Diagrama del flujo secuencial de datos y ciclo de vida de la información a través de las tres capas del sistema.	16
4.1	Estructura de archivos y módulos del <i>software</i> desarrollado para el nodo <i>Edge</i>	22
4.2	Diagrama de clases de las entidades principales del dominio para la persistencia local en <i>IsarDB</i>	25
4.3	Diagrama de flujo del recorrido de árboles y clasificación probabilística en la clase <i>DynamicRandomForest</i>	27
4.4	Interfaz de usuario según la resolución del dispositivo. Barra inferior vs. Riel lateral.	28
4.5	Vista del Menú Principal detallando el indicador de enlace, la tarjeta de recomendación agronómica y la salida de consola. Esta figura muestra tres casos: 1. Conexión con estación IoT sin transacciones iniciadas. 2. Estimación del riego según el tiempo actual, en este caso “riego necesario”. 3. Estimación del riego según la última información guardada en la estación, por lo que la logica del sistema emula la predicción bajo dichas condiciones, en este caso “riego evitable” para el 4/8/2026.	29
4.6	Interfaz del Menú de Conexión <i>Bluetooth</i> con listado de nodos descubiertos y niveles de Received Signal Strength Indicator (RSSI), formulario de emparejamiento, y conexión exitosa.	30
4.7	Pantalla del Menú de Almacenamiento con panel de diagnóstico de API <i>Fog</i> , listado unificado de estaciones con etiquetas LOCAL DB / FOG-SERVER y submenú de inferencia.	31
4.8	Menú de Configuración mostrando las tarjetas de entorno, diagnóstico de red, ventana agronómica y el catálogo de modelos <i>Random Forest</i>	33
5.1	Diagrama de estados del ensamblador de fragmentos binarios BLE en la aplicación móvil.	41
6.1	Arquitectura del servidor y contenedores Docker.	44
6.2	Diagrama Entidad-Relación del motor de base de datos SQLite. Se muestran las tablas de negocio principales, los tipos de datos en la sintaxis de SQLite, las claves principales <i>Primary Key</i> (PK), las relaciones por clave foránea <i>Foreign Key</i> (FK), la cardinalidad y las restricciones de unicidad <i>Unique</i> (UQ) correspondientes a cada tabla.	45
7.1	Esquema detallado de la fase de preprocesamiento, imputación, escalado y fusión temporal del vector de características proveniente del nodo IoT y la API meteorológica.	52
8.1	Comparativa de consumo de recursos en el dispositivo móvil. <i>Edge Computing</i> frente a un esquema dependiente de la nube <i>Cloud Computing</i>	56

Índice de tablas

2.1	Comparativa técnica de tecnologías de transmisión inalámbrica en agricultura.	7
5.1	Comparativa de codesarrollo: diferencias entre la versión original del <i>firmware</i> y las aportaciones realizadas por este Trabajo de Fin de Máster (TFM). Cada limitación técnica en la capa de comunicación y decodificación fue resuelta para habilitar la operación cooperativa con la aplicación móvil en el <i>Edge</i>	36
5.2	Mapa de UUIDs para el servicio primario y las características GATT del nodo de campo.	38
6.1	Relación de <i>endpoints</i> expuestos por la API RESTful del servidor <i>Fog</i> . Se destaca el verbo HTTP, la ruta, la funcionalidad principal y la obligatoriedad del esquema de seguridad.	46
7.1	Resultados empíricos de latencia sobre las distintas aproximaciones a ejecución del modelo <i>Random Forest</i> integrado en el sistema.	53

CAPÍTULO 1

Introducción

El presente TFM está centrado en la convergencia entre la ingeniería de sistemas informáticos, la inteligencia artificial y la sostenibilidad medioambiental. En un contexto global marcado por la escasez hídrica y la necesidad inminente de optimizar los recursos naturales, la digitalización del sector agrícola emerge como una respuesta tecnológica indispensable. Este primer capítulo tiene como propósito asentar las bases del proyecto. A lo largo de las siguientes secciones, exponemos la motivación que impulsa esta investigación, delimitamos la problemática actual de las infraestructuras tradicionales basadas exclusivamente en la nube y definimos de manera precisa los objetivos que guían el desarrollo de la arquitectura descentralizada de *Edge y Fog Computing* propuesta. Finalmente, detallamos la filosofía de diseño adoptada y la estructura organizativa de esta memoria.

1.1 Motivación

La escasez crónica de recursos hídricos, agravada por la aceleración del cambio climático, representa uno de los mayores desafíos para la seguridad alimentaria mundial en el siglo XXI. En regiones de alta vulnerabilidad climática, la optimización del uso del agua en la agricultura ha dejado de ser una opción de mejora económica para convertirse en una necesidad imperativa de supervivencia operativa.

La Agricultura 4.0 y los paradigmas de riego inteligente prometen maximizar el rendimiento de los cultivos minimizando el desperdicio hídrico. Sin embargo, la adopción real de estas tecnologías en zonas de minifundio está obstaculizada por barreras socioeconómicas y tecnológicas: altos costes de inversión, infraestructuras complejas y plataformas cerradas que privan al agricultor de la soberanía sobre sus propios datos. En este contexto, surge la motivación de esta tesis: democratizar el acceso a la agricultura de precisión mediante el uso de herramientas de consumo universales, como los teléfonos inteligentes *smartphones*, transformándolos en potentes nodos de decisión agronómica.

1.2 Problemática

El estado del arte actual en sistemas de gestión agrícola depende predominantemente de arquitecturas centralizadas en la nube. Estas arquitecturas asumen que los nodos de campo son meros recolectores de datos que envían telemetría continua a servidores remotos donde se ejecuta la inteligencia artificial.

Este enfoque presenta deficiencias estructurales severas en entornos rurales:

- **Vulnerabilidad ante la conectividad:** La cobertura celular en zonas agrícolas suele ser intermitente. Una caída de red interrumpe el ciclo predictivo y deja a la explotación ciega ante el estrés hídrico.
- **Ineficiencia energética:** La transmisión constante de datos crudos por radiofrecuencia (redes celulares o LPWAN) agota rápidamente las baterías de los nodos, elevando los costes operativos.
- **Rigidez tecnológica:** Los escasos sistemas que intentan mover la lógica al microcontrolador suelen requerir recompilaciones completas del *firmware* para ajustar un simple umbral agronómico, careciendo de la flexibilidad necesaria frente a cambios estacionales.

1.3 Objetivos

1.3.1. Objetivo General

Diseñar, desarrollar y validar una arquitectura de *Edge/Fog Computing* descentralizada para la optimización del riego agrícola, trasladando la ejecución de modelos de aprendizaje automático (ML y Deep Learning (DL)) al borde de la red, es decir, microcontroladores y *smartphones*. El propósito es eliminar la dependencia crítica de la nube, reducir el consumo energético de la telemetría y garantizar la privacidad y soberanía de los datos del agricultor.

1.3.2. Objetivos Específicos

Para alcanzar el objetivo general, planteamos las siguientes metas técnicas:

1. **Orquestación telemétrica en el borde:** Implementar un protocolo de comunicación robusto y de bajo consumo mediante BLE y Concise Binary Object Representation (CBOR), integrando rutinas de reensamblado asíncrono para ingestar grandes volúmenes de datos desde microcontroladores restringidos.
2. **Desarrollo de nodo táctico *Edge*:** Construir una aplicación móvil multiplataforma con arquitectura *offline-first* que actúe como centro de mando, fusionando la telemetría edáfica con pronósticos macroclimáticos mediante APIs externas.
3. **Inferencia dinámica *On-Device*:** Diseñar e integrar un motor predictivo dual en el *smartphone* capaz de ejecutar redes Long-Short Term Memory (LSTM) y árboles de decisión dinámicos sin requerir conexión a internet en el momento de la inferencia.
4. **Gestión *Fog* y soberanía de datos:** Desarrollar un servidor central auto-alojable que incluya un microservicio de conversión (rf2json) para compilar y distribuir modelos estadísticos en caliente hacia los terminales móviles.
5. **Validación computacional y prueba de concepto:** Evaluar la arquitectura en un entorno de laboratorio mediante inyección de telemetría simulada, demostrando un uso de recursos eficiente en el dispositivo móvil y validando conceptualmente la toma de decisiones agronómicas frente a los sistemas de riego basados en umbrales rígidos.

1.4 Propuesta y Filosofía Arquitectónica

Para resolver la problemática expuesta, este trabajo propone un enfoque de co-diseño híbrido basado en la descentralización del cómputo. La infraestructura propuesta descarga a los microcontroladores de campo de tareas pesadas de red, delegando la agregación y la inferencia predictiva principal al *smartphone* del operario.

Para mantener una huella computacional mínima y atenernos estrictamente a principios de ingeniería ágil, el desarrollo se ha regido por el paradigma *You Aren't Gonna Need It*. Bajo esta premisa, hemos prescindido intencionalmente de *frameworks* de terceros para la inferencia de *Machine Learning* en la aplicación móvil, optando en su lugar por motores iterativos propios y estructuras de datos nativas. Esto no solo garantiza un rendimiento superior y un peso mínimo de la aplicación, sino que preserva el control total sobre la lógica del ciclo de vida del dato bajo los principios de *Privacy by Design*.

1.5 Estructura de la Memoria

El presente documento se estructura de la siguiente manera:

- **Capítulo 2:** Establece el marco teórico, abordando la agricultura de precisión, las redes Internet of Things (IoT) y los paradigmas de *Edge Computing*.
- **Capítulo 3:** Define la arquitectura global del sistema, sus capas lógicas y el flujo de la información.
- **Capítulo 4:** Desglosa el desarrollo de software de la aplicación móvil, justificando las decisiones de interfaz y persistencia.
- **Capítulo 5:** Detalla la capa física, los desafíos del codesarrollo del *firmware* y la integración asíncrona del protocolo BLE.
- **Capítulo 6:** Expone la construcción del servidor *Fog* auto-alojable y la estrategia de distribución de modelos.
- **Capítulo 7:** Describe la orquestación del *Machine Learning* y la ejecución del *pipeline* predictivo.
- **Capítulo 8:** Presenta la evaluación cuantitativa y la prueba de concepto en entorno de laboratorio mediante inyección de telemetría simulada.
- **Capítulo 9:** Recoge las conclusiones del proyecto y plantea futuras líneas de investigación.

CAPÍTULO 2

Análisis y Fundamentos

Para comprender y justificar en profundidad las decisiones arquitectónicas y tecnológicas adoptadas en el diseño de este sistema, resulta fundamental establecer previamente el marco teórico y analizar el estado del arte. Este segundo capítulo explora los pilares conceptuales y técnicos que sustentan el proyecto.

En primer lugar, contextualizamos el dominio de aplicación a través de la agricultura de precisión y el papel crítico de las redes de sensores inalámbricos de bajo coste para la captura de telemetría en campo. A continuación, examinamos las debilidades del paradigma tradicional basado en el *Cloud Computing* frente a las ventajas operativas de la descentralización mediante *Edge Computing*. Por último, abordamos los fundamentos de los algoritmos de *Machine Learning* y las tecnologías de desarrollo móvil multiplataforma que habilitan la orquestación y la inferencia predictiva directamente en el borde de la red, sentando así la base teórica para la propuesta desarrollada en los capítulos posteriores.

2.1 Agricultura de Precisión

Como adelantábamos en la presentación de esta investigación, el despliegue de redes de sensores y la inferencia predictiva en el borde proponen un nuevo paradigma descentralizado, diseñado para operar bajo restricciones severas de energía y conectividad. Si bien esta arquitectura es aplicable a múltiples verticales del IoT, su integración resulta especialmente crítica y transformadora en escenarios donde la toma de decisiones en tiempo real frente a condiciones ambientales cambiantes es vital. Uno de los sectores que mayor urgencia presenta para la adopción de estos sistemas autónomos es la agricultura de precisión.

La realidad del siglo XXI presenta un escenario sin precedentes ante la humanidad moderna. El cambio climático global acelerado y escasez crónica de recursos hídricos es un hecho de alcance mundial. El sector agrícola es altamente afectado, de forma que maximizar el rendimiento de los cultivos bajo estas condiciones cada vez más extremas se ha convertido en uno de los principales objetivos de optimización.

En regiones de alta vulnerabilidad, como la cuenca del Mediterráneo y de manera acentuada en España, la variabilidad climática se manifiesta en periodos de sequía más severos, prolongados e impredecibles, acompañados de un incremento en las tasas de evapotranspiración potencial. Este fenómeno ejerce una presión sobre las masas de agua subterránea y los sistemas de embalses regulados. La gestión hídrica convencional, basada en calendarios fijos de riego o en la simple intuición del agricultor, se transforma en un método ineficiente y peligroso para la sostenibilidad del ecosistema. En este contexto, la transformación digital del sector agrícola a través de la digitalización del riego surge

no solo como una opción de optimización de costos a medio-largo plazo, sino como una necesidad imperativa de resiliencia operativa para garantizar la seguridad alimentaria y la viabilidad económica de las explotaciones.

A pesar de los evidentes beneficios teóricos de la agricultura de precisión, su despliegue comercial y práctico en el territorio español se enfrenta a complejas barreras de carácter socioeconómico y estructural. En primer lugar, la estructura de la propiedad de la tierra en diversas comunidades autónomas se caracteriza por el minifundio y una elevada fragmentación parcelaria, lo que diluye las economías de escala necesarias para amortizar grandes infraestructuras tecnológicas centralizadas [2]. En segundo lugar, la demografía del sector agrícola presenta un marcado envejecimiento de la población activa, lo que se traduce en una brecha digital sustancial y una resistencia cultural al uso de interfaces complejas o sistemas que operen de forma autónoma como una “caja negra” [11, 10].

Desde la perspectiva financiera, la adopción se ve fuertemente penalizada por los altos costos de inversión inicial¹, que engloban la adquisición de sensores con grado industrial, pasarelas de comunicación y actuadores automatizados. A esto se suman los costos de mantenimiento operativo² asociados al reemplazo de baterías, calibración periódica de sondas en campo y el pago de suscripciones recurrentes a plataformas en la nube. Frente a estos desembolsos, los retornos de inversión suelen ser inciertos a corto plazo en cultivos de gran extensión o bajo valor comercial unitario, lo que frena la toma de decisiones financieras [11, 10]. Esta realidad demuestra la necesidad de desarrollar soluciones de bajo costo y arquitecturas de transición digital basadas en herramientas familiares, como el teléfono inteligente, que actúen como un soporte a la decisión manteniendo la autonomía supervisada del factor humano, también conocido como *human-in-the-loop*.

El diseño de un sistema de riego inteligente requiere un modelado riguroso de la interacción suelo-planta-atmósfera. El núcleo conceptual para determinar la demanda de agua es la estimación de la evapotranspiración de referencia calculada mediante métodos normalizados como la ecuación de Penman-Monteith de la FAO, la cual integra variables meteorológicas críticas: radiación solar neta, temperatura del aire, velocidad del viento y déficit de presión de vapor. Para un análisis detallado sobre este modelado, remitimos al trabajo de [8] en fecha de su publicación.

No obstante, la optimización hídrica no debe limitarse a la reposición volumétrica del agua transpirada, sino que debe equilibrarse con la dinámica de la patología vegetal. Un aporte hídrico desmedido o ejecutado en franjas horarias inadecuadas prolonga la duración de la humedad foliar en la superficie del suelo y el dosel de la planta. Esto genera microclimas saturados de humedad relativa que disparan el riesgo de germinación de esporas e infecciones fúngicas causadas por fitopatógenos endémicos³. Por ende, el algoritmo de riego debe computar el balance hídrico del suelo de forma predictiva, integrando sensores de humedad de la biomasa foliar y predicciones meteorológicas a corto plazo para mitigar proactivamente las condiciones propicias para las enfermedades fúngicas.

2.1.1. Redes de Sensores Inalámbricos de Bajo Costo

Para alimentar los modelos agronómicos con datos de alta resolución espacio-temporal, el estado del arte ha desplazado los métodos de muestreo manual tradicionales por las Redes de Sensores Inalámbricos. Estas arquitecturas permiten el despliegue distribuido

¹CAPEX

²OPEX

³*Botrytis cinerea*, mildiu u oídio

de múltiples nodos sensores autónomos y de bajo costo energético en puntos estratégicos de la parcela agrícola.

El principio fundamental de una red inalámbrica de sensores agrícola radica en la monitorización y adquisición continua en campo. Cada nodo periférico se diseña con microcontroladores de ultra bajo consumo que permanecen en modos de suspensión profunda⁴, despertando cíclicamente solo para energizar los sensores, capturar las señales analógicas o digitales, y transmitir el paquete de datos hacia la pasarela central o nodo coordinador cuando sean las condiciones apropiadas. Luego, retorna inmediatamente al estado de ahorro energético para maximizar la vida útil de sus baterías.

La conectividad en entornos rurales constituye uno de los mayores desafíos de la Agricultura 4.0. Las tecnologías de transmisión inalámbrica utilizadas en las redes inalámbrica de sensores deben equilibrar tres restricciones críticas: alcance geográfico, consumo energético y ancho de banda. En la Tabla 2.1 se puede ver un resumen considerando estos puntos [1].

Tecnología	Alcance Típico	Consumo Energético	Ancho de Banda	Dependencia de Infraestructura
ZigBee	100 m - 1 km	Muy Bajo	250 kbps	Nula
LoRaWAN	5 km - 15 km	Ultra Bajo	50 kbps	Privada o Pública
NB-IoT	Celular	Bajo	250 kbps	Alta
BLE	<100 m	Ultra Bajo	2 Mbps	Nula

Tabla 2.1: Comparativa técnica de tecnologías de transmisión inalámbrica en agricultura.

Mientras que protocolos como *ZigBee* ofrecen excelentes topologías en malla pero adolecen de un alcance limitado ante la atenuación geométrica provocada por el follaje denso, las tecnologías LPWAN (*Low-Power Wide-Area Network*) como LoRaWAN se han consolidado por su extraordinaria inmunidad al ruido y capacidad de cubrir radios kilométricos en línea de vista gracias a su modulación *Chirp Spread Spectrum* [1].

Por su parte, alternativas licenciadas como NB-IoT eliminan la necesidad de desplegar pasarelas propias al conectarse directamente a las torres de telefonía celular existentes, aunque imponen una dependencia estricta de la cobertura del operador y un costo por tarjeta SIM. La convergencia con redes 5G abre la puerta al manejo masivo de dispositivos conectados, conectando miles de sensores por hectárea con latencias insignificantes.

Finalmente, en el extremo opuesto del espectro de alcance, destacan protocolos de proximidad como BLE. Aunque su cobertura geográfica está severamente restringida a unas pocas decenas de metros, esta tecnología está posicionada como el estándar ideal para la interacción directa *in situ*. Su ultra bajo consumo energético, sumado a un ancho de banda muy superior al de las alternativas LPWAN, habilita descargas masivas y rápidas de registros históricos acumulados. Precisamente por su ubicuidad en teléfonos inteligentes comerciales y su capacidad para orquestar transferencias de telemetría sin depender de infraestructuras de terceros, BLE es la tecnología elegida en este trabajo para comunicar el hardware sensor de campo *Deep-Edge* con la aplicación *Edge*.

2.1.2. Calidad y Fiabilidad del Dato

Los entornos agrícolas imponen condiciones ambientales extremas, como choques térmicos, humedad de saturación, radiación UV incidente, corrosión galvánica por fertilizantes... que degradan físicamente el hardware sensor. En consecuencia, la pérdida de calibración, el agotamiento de baterías o los fallos de hardware introducen anomalías en la captura de la telemetría, tales como valores atípicos, ruidos de alta frecuencia o pér-

⁴*Deep Sleep*

didadas prolongadas de señal. Si estos datos corruptos alimentan directamente un modelo predictivo de ML, la salida provocará veredictos de riego erróneos, con un consecuente estrés hídrico o desperdicio masivo de agua.

Para garantizar la calidad y fiabilidad del dato, el estado del arte exige el desarrollo de estrategias de validación y curación en las primeras etapas del pipeline de datos. Esto incluye la ejecución de algoritmos adaptativos de detección de anomalías⁵ implementados en la propia capa física/sensor o en el gateway de borde. Estos algoritmos no solo descartan los datos erróneos mediante umbrales físicos duros y lógicos, sino que reconstruyen la serie temporal en tiempo real mediante técnicas ligeras de interpolación o imputación lineal y espacial, aislando al sistema inteligente de los fallos inherentes al hardware expuesto a la intemperie.

2.2 *Edge Computing y Cloud Computing*

Durante la última década, los sistemas de gestión agrícola inteligente han dependido predominantemente de arquitecturas centralizadas basadas en la nube⁶. Bajo este paradigma, los nodos sensores periféricos actúan de manera pasiva, limitándose a capturar variables físicas e iniciar transmisiones masivas de telemetría a través de redes Wireless Area Network (WAN) hacia servidores remotos donde se ejecutan los algoritmos de decisión y optimización hídrica. No obstante, el despliegue en condiciones reales de producción, particularmente en los campos de cultivo de la península ibérica, evidencia limitaciones estructurales críticas inherentes a este enfoque centralizado.

El primer cuello de botella radica en la dependencia crítica de una inestable conectividad de banda ancha rural. La cobertura de redes celulares en zonas agrícolas alejadas de los núcleos urbanos suele ser intermitente, registrando pérdidas frecuentes de paquetes y desafecciones de red que interrumpen el flujo de datos. Si el *pipeline* predictivo se ejecuta exclusivamente en la nube, un apagón de red impide que la explotación reciba las órdenes de riego calculadas, comprometiendo de inmediato la integridad del cultivo.

En segundo lugar, la transmisión continua y sin procesar de grandes volúmenes de telemetría hacia la nube penaliza severamente el balance energético de los nodos de campo, dado que los transceptores de radio integrados representan el mayor vector de consumo de batería [1].

Por último, la arquitectura puramente centralizada introduce problemas de latencia e ineficiencia económica, debido al costo operativo recurrente asociado al ancho de banda consumido y al almacenamiento de datos crudos redundantes en repositorios en la nube.

Para mitigar las vulnerabilidades del almacenamiento y cómputo centralizados, el estado del arte propone la descentralización de la inteligencia mediante el paradigma del procesamiento local, estructurado a través de capas de *Edge Computing* y *Fog Computing*. En este ecosistema, las tareas de cómputo, filtrado y toma de decisiones se aproximan físicamente al origen del dato, como puede ser la parcela agrícola, transformando las pasarelas locales y los microcontroladores de campo en nodos activos con capacidad de procesamiento.

La tesis esquematizada en la Figura 2.1 implementa un enfoque de co-diseño híbrido donde la carga de trabajo computacional es distribuido estratégicamente. En la capa de instrumentación de campo, arquitecturas basadas en microcontroladores de bajo costo y alto rendimiento asumen las tareas de menor nivel: agregación del dato en ventanas de tiempo deslizando, filtrado físico de ruido eléctrico, preprocesamiento de un pronóstico

⁵Basados en filtros de Kalman, medias móviles exponenciales o ventanas rolling de desviación estándar.

⁶*Cloud Computing*

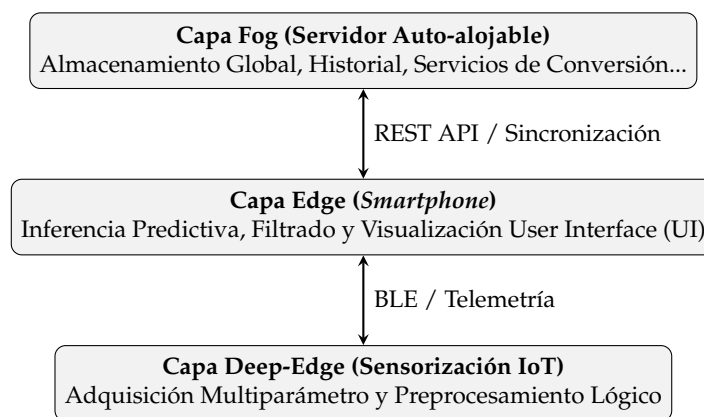


Figura 2.1: Diagrama conceptual de la arquitectura de co-diseño híbrido.

basado en series temporales y lógica de control adaptativo. Por encima de esta capa se sitúa el teléfono inteligente (*smartphone*) actuando como un centro logístico táctico del borde. Este terminal móvil no solo se comunica bidireccionalmente con los nodos IoT mediante protocolos de proximidad como BLE, sino que asume de manera local la ingesta de telemetría de la estación y su fusión con datos meteorológicos recopilados de APIs de internet [3].

Al estructurar esta topología jerárquica, la dependencia de la infraestructura en la nube es desplazada a tareas no críticas en tiempo real, tales como el almacenamiento histórico agregativo a largo plazo y la sincronización multi-inquilino para la colaboración entre agricultores, además de servicios de gestión de modelos predictivos sin necesidad de actualización constante del sistema.

La evolución más disruptiva dentro del procesamiento de borde es el surgimiento de la Inteligencia Artificial Descentralizada o *Edge AI*. Este paradigma consiste en trasladar los modelos matemáticos y predictivos de ML, tradicionalmente restringidos a servidores, hacia entornos de hardware con recursos computacionales severamente limitados. La viabilidad técnica de esta aproximación está fundamentada en el uso de técnicas modernas de optimización de modelos, como la cuantización de pesos a precisión entera de 8 bits, permitiendo que la inferencia local opere con huellas de memoria RAM y almacenamiento flash despreciables.

2.3 Machine Learning

Tradicionalmente, la automatización del riego se ha fundamentado en sistemas de control basados en reglas fijas y umbrales rígidos de humedad, por ejemplo:

Activar la válvula si el valor del contenido volumétrico de agua desciende del 20 %.

Si bien este enfoque reactivo supuso un avance frente al riego manual, presenta deficiencias severas: ignora la inercia térmica del suelo, la evapotranspiración futura y el estadio fenológico⁷ del cultivo, provocando con frecuencia fenómenos de percolación profunda y lixiviación⁸ de nutrientes. El estado del arte actual evidencia una transición definitiva hacia el control predictivo y adaptativo. La integración de técnicas de ML permite a los

⁷Estudio de los fenómenos biológicos en relación con el clima, particularmente en los cambios estacionales.

⁸Tratar una sustancia compleja, como un mineral, con un disolvente adecuado para separar sus partes solubles de las insolubles.

sistemas aprender de los patrones históricos de la interacción suelo-planta-atmósfera, anticipándose al déficit hídrico antes de que este induzca un estrés fisiológico en la planta [3].

Para la predicción de variables dinámicas complejas, destacamos la implementación de algoritmos de aprendizaje supervisado clásico. Debido a la naturaleza no lineal de los procesos agrometeorológicos, modelos como Árboles de Decisión (*Decision Trees*) y Máquinas de Vectores de Soporte (SVM) han demostrado una eficacia superior a las regresiones estadísticas tradicionales. En escenarios con múltiples variables (temperatura foliar, radiación solar, humedad relativa), los métodos de ensamblaje como *Random Forest*, *XGBoost* y *Gradient Boosting* posicionados como el estándar de la industria. Estos algoritmos mitigan el sobreajuste y ofrecen una alta interpretabilidad a través del análisis de la importancia de las características, permitiendo al agrónomo comprender qué variables climáticas están impulsando la decisión del modelo.

El comportamiento del agua en el suelo no es un evento estático, sino un proceso fuertemente dependiente del tiempo. Por ello, el modelado temporal avanzado recurre a arquitecturas de DL, específicamente las Redes Neuronales Recurrentes con memoria a corto y largo plazo (LSTM).

Las redes LSTM son excepcionalmente competentes para capturar dependencias temporales prolongadas, como la curva de decaimiento de la humedad del suelo tras un evento de lluvia o la estimación de la recarga del nivel freático subterráneo.

Sin embargo, para que estos modelos generen calendarios de riego óptimos, los datos históricos no son suficientes. El estado del arte exige la fusión de datos multimodales: las series temporales de la telemetría local deben combinarse dinámicamente con predicciones meteorológicas a corto plazo consumidas en tiempo real a través de APIs climáticas externas. Esta fusión permite al algoritmo suspender un evento de riego programado si la API pronostica los índices de evaporación con alta confianza, buscando maximizar el ahorro hídrico solo cuando sea permitible. Algunos proyectos que han seguido una línea de desarrollo relacionada son [9, 12].

2.4 Arquitecturas Móviles para *Edge Computing*

El despliegue de las tecnologías mencionadas requiere una infraestructura de software capaz de orquestar el flujo de datos. En el contexto de la Agricultura 4.0, el *smartphone* está consolidado no solo como un terminal de visualización de telemetría, sino como el nodo táctico principal de *Edge Computing* [3].

Esta aproximación facilita la ejecución de esquemas de autonomía supervisada *human-in-the-loop*, donde la inteligencia artificial recomienda un veredicto de riego, pero el agricultor retiene la autoridad final de ejecución, fomentando la confianza y la adopción tecnológica. Para agilizar el desarrollo y mantener un rendimiento cercano al nativo en múltiples ecosistemas (iOS, Android y Linux), la literatura contemporánea respalda el uso de tecnologías de desarrollo multiplataforma como el *SDK Flutter* y su lenguaje subyacente, *Dart*.

La complejidad inherente de una aplicación que gestiona conexiones IoT, bases de datos locales e inferencia de modelos predictivos exige una arquitectura de software robusta. El estado del arte en ingeniería de software móvil prioriza una estructura de código limpia y fuertemente desacoplada. Esto es logrado mediante la organización modular del código fuente en directorios orientados a dominios o entidades independientes, lo que garantiza la extensibilidad y escalabilidad del proyecto.

La gestión del ciclo de vida de la aplicación y la inyección de dependencias son manejadas de forma declarativa mediante gestores de paquetes, como `pubspec.yaml` en el ecosistema Flutter. Hemos buscado aplicar patrones que aíslan la capa de lógica de negocio de la capa de presentación, asegurando que la interfaz se reconstruya reactivamente en tiempo real sin bloquear el hilo principal de ejecución del dispositivo.

La distribución del cómputo en arquitecturas de borde es un área de intenso debate académico. Reportes técnicos distinguen claramente los roles: los microcontroladores de campo deben limitarse a ejecutar lógica ligera de adquisición y filtrado primario para conservar batería. Por el contrario, los dispositivos móviles, equipados con procesadores *multicore* y aceleradores de *hardware*, asumen la carga de visualización, gestión de interfaces y, fundamentalmente, la inferencia predictiva embebida.

No obstante, un análisis sobre el estado del arte revela una brecha de *frameworks* en producción. Existe un desfase técnico entre el desarrollo científico que publica constantemente avances sobre la eficiencia computacional de librerías como *TensorFlow Lite* o *PyTorch Mobile* en dispositivos móviles, y la realidad comercial del sector agrícola. Las plataformas comerciales de riego aún registran una adopción incipiente de la inferencia genuinamente embebida, prefiriendo anclarse en esquemas híbridos fuertemente dependientes de la nube. Este trabajo de investigación aborda directamente esta brecha, demostrando la viabilidad técnica y operativa de trasladar la ejecución integral del modelo predictivo al *smartphone*, eliminando la latencia de red y garantizando la resiliencia del sistema en entornos rurales aislados [12, 7].

CAPÍTULO 3

Arquitectura Global del Sistema

Este capítulo constituye el eje vertebrador de nuestra memoria técnica, donde articulamos nuestra visión en ingeniería de sistemas empujados, redes de comunicaciones y computación distribuida. En las siguientes secciones justificamos nuestra decisión de abandonar el paradigma centralizado en la nube *Cloud-Centric* para adoptar una topología jerárquica orientada al procesamiento en el borde *Edge Computing*.

Concebimos la arquitectura global del sistema como una solución híbrida y distribuida, diseñada específicamente para responder a las restricciones operativas del entorno agrícola peninsular: una infraestructura de conectividad WAN intermitente, recursos energéticos severamente limitados en campo y la necesidad crítica de garantizar al agricultor la soberanía sobre sus datos agronómicos sin incurrir en costes por plataformas comerciales externas.

Para articular este flujo y asegurar una delimitación clara de responsabilidades computacionales, hemos estructurado la topología de red y procesamiento en tres capas físicas y lógicas interconectadas:

1. **Capa de Instrumentación y Adquisición Física** | *Nodos Deep-Edge*: Constituye el nivel más periférico de nuestra red y está materializada por estaciones agroclimáticas autónomas portadoras del *firmware* embebido *Savia*, desarrollado por [4]. Operamos estas estaciones sobre la plataforma *Raspberry Pi Pico 2 W*, integrando el sistema en chip RP2350. Al diseñar estos nodos, priorizamos la máxima eficiencia energética y resiliencia mediante ciclos de suspensión profunda periódicos y rutinas de adquisición temporizadas.

A nivel funcional, el *firmware* embebido opera de forma determinista asumiendo cuatro responsabilidades concurrentes:

- **Adquisición y Sensorización:** Establece una interfaz digital directa con instrumentación agrícola especializada¹ mediante el protocolo multipunto estándar SDI-12.
- **Agregación y Persistencia Local:** Preprocesa y filtra las series temporales brutas para consolidar medias horarias. Custodia temporalmente la información en estructuras de búfer circular sobre memoria no volátil Flash, asegurando una retención mínima de 48 horas ante caídas prolongadas del enlace inalámbrico.
- **Inferencia sobre el Dispositivo:** Aprovecha la unidad de coma flotante (FPU) y los 520 KB de memoria SRAM del microcontrolador RP2350 para ejecutar localmente modelos de aprendizaje profundo. Específicamente, ejecuta redes

¹Sondas de humedad volumétrica del suelo y temperatura edáfica.

neuronales recurrentes LSTM cuantizadas ² mediante *TensorFlow Lite for Microcontrollers*, dotando a la estación de autonomía para generar pronósticos a 24 horas sobre el estado hídrico sin conectividad externa.

- **Interfaces de Comunicación:** Expone la telemetría histórica y los vectores de predicción hacia la capa intermedia a través de perfiles BLE Generic Attribute Profile (GATT). Adicionalmente, dispone de un canal de largo alcance y baja potencia basado en tecnología LoRa para la emisión periódica de tramas de estado crítico cuando la cobertura del transceptor lo permite.

2. **Capa Logística y de Inferencia** | *Software Edge*: Materializamos esta capa a través del teléfono inteligente del operario agrícola mediante una aplicación nativa desarrollada en el entorno *Flutter/Dart*. En nuestra topología, asignamos al terminal móvil el rol activo de *Cuartel General Logístico* y motor principal de ejecución táctica:

- **Ingestión de Datos en Proximidad:** Gestionamos el descubrimiento y la sincronización proactiva de corto alcance con los nodos *Deep-Edge*. La aplicación ingesta la telemetría histórica acumulada a través de enlaces asíncronos BLE, garantizando la recolección de datos a pie de parcela sin requerir cobertura celular.
- **Fusión de Datos y Contexto Climático:** Enriquecemos las lecturas físicas de las sondas con el contexto macroclimático exterior. Para ello, realizamos consultas asíncronas hacia la plataforma Open-Meteo vía red celular WAN, fusionando localmente las variables microclimáticas en tiempo real con las previsiones meteorológicas a corto plazo.
- **Inferencia Predictiva en el Edge:** Evaluamos las predicciones localmente mediante un esquema dual optimizado. Por un lado, incorporamos en la aplicación una réplica algorítmica nativa en Dart del modelo predictivo LSTM para proyectar la curva de humedad del suelo a 24 horas cuando la estación de campo opera en modo *forward* (con la inferencia local desactivada en el microcontrolador). Por otro lado, implementamos un motor desacoplado para modelos *Random Forest*, cuyos árboles de decisión recibimos serializados en formato JSON desde la capa superior y reconstruimos dinámicamente en tiempo de ejecución. Este diseño nos permite actualizar en caliente la lógica agronómica sin necesidad de recompilar la aplicación móvil.
- **Pasarela de Sincronización:** Empleamos la aplicación como puente lógico bidireccional hacia la capa superior *Fog*. Al detectar una conexión a Internet estable, canalizamos la subida asíncrona de las series temporales consolidadas hacia la base de datos central y descargamos actualizaciones de los modelos predictivos.

3. **Capa de Consolidación y Sincronización Global** | *Servidor Fog Auto-Alojable*: Diseñamos este eslabón de agregación a nivel de finca o cooperativa para materializar el paradigma de *Fog Computing*. Bajo una filosofía estricta de soberanía de datos, desarrollamos un servidor telemático ligero y auto-alojable sobre el entorno de ejecución *Node.js*, encargado de:

- **Persistencia y Cooperación Distribuida:** Mantenemos un repositorio maestro unificado basado en un motor de base de datos relacional y transaccional *SQLite*, con el que facilitamos la sincronización asíncrona de los datos recolectados por múltiples terminales móviles y habilitamos redes de cooperación compartida entre agricultores.

²El mismo modelo cuantizado sin pérdidas que afectan a la confiabilidad analítica lo reutilizamos posteriormente para la inferencia en la aplicación de usuario.

- **Distribución de Modelos Predictivos:** Gestionamos y versionamos modelos *Random Forest* convertidos a esquemas JSON, permitiendo a los nodos *Edge* actualizar sus parámetros estacionales de decisión de forma transparente.
- **Seguridad y Despliegue Aislado:** Canalizamos todas las transacciones mediante una Application Programming Interface (API) RESTful segura con autenticación estricta, limitación de tasa de peticiones en memoria (*Rate Limiting*) y un despliegue modular contenerizado mediante *Docker* [6] bajo perfiles de usuario sin privilegios.

Con esta distribución estratégica optimizamos el uso del ancho de banda y la vida útil de las baterías en los nodos empotrados, dotando al sistema de una elevada tolerancia a fallos. La pérdida total de conectividad con la capa *Fog* no nos impide recolectar datos de campo vía BLE en el terminal móvil ni inferir localmente las necesidades hídricas del cultivo [2].

3.1 Logística

En nuestra arquitectura híbrida y descentralizada, concebimos el teléfono inteligente como el núcleo de orquestación y decisión táctica. Asignamos a este dispositivo la responsabilidad de articular el flujo de información asíncrono entre los nodos de instrumentación dispersos, las plataformas climáticas externas y la infraestructura de almacenamiento global mediante la coordinación de tres áreas técnicas:

1. **Orquestación del Enlace y Descubrimiento en Proximidad (BLE):** Mediante rutinas de escaneo en segundo plano, ejecutamos un proceso activo de descubrimiento de estaciones agroclimáticas *Savia*. Con el fin de minimizar el ciclo de trabajo de la antena y evitar la sobrecarga del hilo principal del sistema operativo, aplicamos en el módulo de radio un filtrado selectivo por el *Service UUID* del perfil GATT de la estación. Una vez validada la proximidad y estimada la estabilidad del enlace mediante el indicador RSSI, negociamos la conexión lógica e iniciamos la descarga secuencial de los búferes de telemetría. Ante pérdidas de enlace debidas a la atenuación del entorno, integramos una máquina de estados finitos que gestiona reintentos exponenciales desacoplados para restaurar el flujo sin intervención del operario.
2. **Gestión de Protocolos Heterogéneos de Red:** Utilizamos la aplicación móvil como un conmutador lógico entre tres pilas de comunicaciones con requisitos físicos dispares:
 - **Capa de Corto Alcance | Deep-Edge:** La operamos sobre BLE mediante lecturas asíncronas y suscripciones a características de notificación GATT, optimizando el ancho de banda mediante tramas binarias compactas.
 - **Capa de Servicios Externos:** La operamos sobre la suite TCP/IP mediante peticiones HTTPS/REST para ingerir previsiones macroclimáticas estructuradas en JSON.
 - **Capa Fog | Servidor Central:** La operamos mediante transacciones RESTful sobre TCP/IP (vía red celular o Wi-Fi) para la transferencia diferida en lotes y la descarga de modelos predictivos actualizados.
3. **Orquestación de la Inferencia Dual y la Persistencia Local:** Al recepcionar las tramas binarias vía BLE, trasladamos las tareas de decodificación y transformación a

hilos de procesamiento secundarios para prevenir caídas de rendimiento en la interfaz gráfica. Almacenamos inmediatamente los registros en la base de datos local no relacional e indexada *IsarDB* bajo una política estricta de funcionamiento sin conexión³. Posteriormente, nuestro motor logístico orquesta el proceso predictivo: unifica la telemetría edáfica más reciente con las variables ambientales externas y ejecuta el modelo temporal LSTM o el clasificador *Random Forest*, emitiendo el veredicto de riego en milisegundos directamente en el campo.

3.2 Flujo de Datos

Definimos el flujo de datos como la trayectoria completa y las sucesivas transformaciones estructurales de la información desde su captura en el estrato físico (*Deep-Edge*) hasta su consolidación transaccional en el servidor central. La Figura 3.1 sintetiza las etapas clave de este cauce de procesamiento distribuido.

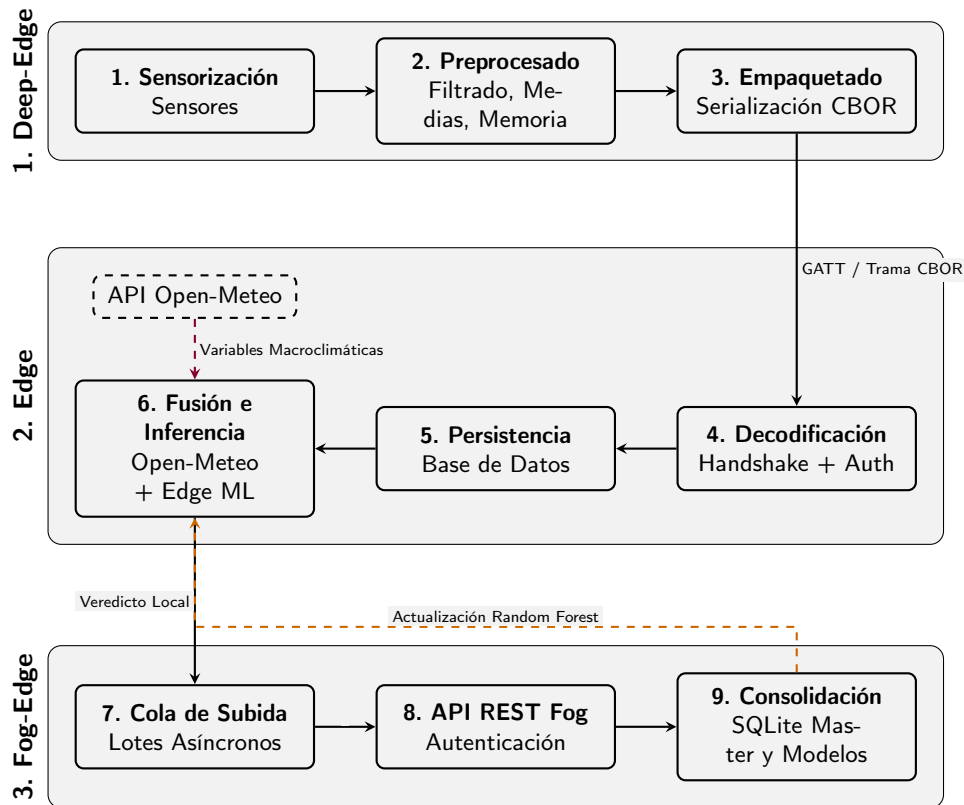


Figura 3.1: Diagrama del flujo secuencial de datos y ciclo de vida de la información a través de las tres capas del sistema.

Formalizamos el ciclo de vida del dato en las siguientes fases operativas:

- 1. Capa 1. Sensorización:** Las sondas edáficas e instrumentación meteorológica convierten los parámetros físicos en señales eléctricas. El microcontrolador, ejecutando el *firmware Savia*, lee los buses de comunicación digital (SDI-12) y las entradas analógicas.
- 2. Capa 1. Preprocesado:** Implementamos algoritmos de filtrado digital para suprimir picos de ruido electromagnético. Conmutamos las muestras tomadas a alta frecuen-

³Offline-first.

cia en promedios o agregaciones horarias y escribimos estas marcas en un búfer circular sobre memoria Flash no volátil, asegurando la preservación del dato durante 48 horas sin enlace inalámbrico.

3. **Capa 1. Empaquetado:** Para reducir drásticamente la sobrecarga de la trama y optimizar el ancho de banda en el canal de radio, codificamos toda la estructura de datos que transmitimos por BLE bajo el estándar CBOR. Esta representación binaria compacta añade una capa de ofuscación nativa frente al análisis de tráfico convencional. Publicamos la trama resultante mediante la característica GATT de notificación BLE.
4. **Capa 2. Decodificación:** La aplicación móvil intercepta la notificación BLE. Antes de procesar la telemetría, ejecutamos un apretón de manos criptográfico por desafío mutuo mediante un hash SHA-256 derivado de un *nonce* aleatorio y una clave precompartida. Tras validar la identidad del nodo, transferimos el *payload* a un hilo secundario que desempaqueta la estructura CBOR.
5. **Capa 2. Persistencia:** Insertamos de inmediato los registros decodificados en la base de datos local no relacional e indexada *IsarDB* [13]. Anclamos así el sistema al paradigma *offline-first*, garantizando que toda la telemetría e historial de decisiones permanezcan en el perímetro digital exclusivo del usuario de forma altamente eficiente en memoria y tiempo de consulta.
6. **Capa 2. Fusión e Inferencia Dual:** En paralelo a la persistencia, realizamos una llamada asíncrona a la API de Open-Meteo vía HTTPS. Obtenemos las coordenadas de forma automática o mediante un selector manual sobre un mapa difuso basado en *OpenStreetMap* para proteger la ubicación exacta. Alineamos la serie temporal local y la predicción macroclimática para alimentar nuestro motor de inferencia dual: ejecutamos la réplica algorítmica local en Dart del modelo LSTM y, de forma complementaria, el evaluador dinámico del modelo *Random Forest* reconstruido a partir de definiciones JSON, generando el veredicto de riego en milisegundos directamente en el terminal.
7. **Capa 3. Sincronización Asíncrona Loteada e Intercambio de Modelos:** Al detectar una red de datos estable, el usuario puede extraer los lotes de registros pendientes desde el almacenamiento local y enviarlos mediante peticiones POST HTTPS autenticadas hacia la API REST de nuestro servidor *Fog* en *Node.js*. El servidor consolida las series temporales en el repositorio maestro *SQLite* y, de forma transparente, provee a la aplicación móvil las actualizaciones de los modelos predictivos *Random Forest* en formato JSON para adaptar la lógica de decisión estacional.

3.3 Seguridad y Privacidad de Datos

Durante el diseño de la arquitectura global, integramos la seguridad y la privacidad desde la fase conceptual bajo los principios de *Security and Privacy by Design*. Al operar en un entorno de infraestructura crítica como la gestión de recursos hídricos agrícolas, consideramos imprescindible neutralizar los vectores de ataque en cada una de las capas expuestas.

3.3.1. Superficie de Ataque y Mitigación en el Enlace Inalámbrico

Las transmisiones de radiofrecuencia no guiadas en campo abierto son vulnerables a ataques de interceptación ⁴, suplantación de identidad del nodo ⁵ e inyección no autorizada de comandos. Para mitigar estos riesgos en el nivel *Deep-Edge*, implementamos en el *firmware* embebido las siguientes medidas:

- **Autenticación Desafío-Respuesta:** En lugar de depender de mecanismos de emparejamiento complejos a nivel de enlace de radio, custodiamos el acceso a la estación *Savia* mediante un esquema de autenticación basado en *Challenge-Response* con funciones de dispersión *SHA-256*.

Al establecer la conexión, el microcontrolador genera un número aleatorio de un solo uso *nonce* de 16 bytes mediante su generador de números aleatorios por hardware (TRNG).

El terminal móvil demuestra el conocimiento de la clave precompartida transmitiendo el resumen criptográfico del *nonce* combinado con dicha clave, el cual verificamos en tiempo constante para mitigar ataques de canal lateral por temporización. Las características GATT de control y configuración permanecen bloqueadas ante cualquier intento de escritura no autenticado.

- **Codificación Compacta y Validación Estricta de *Payloads*:** Estructuramos la comunicación de datos y configuración mediante el estándar binario CBOR [5]. Nuestro decodificador embebido valida estrictamente los tipos, longitudes de campos y límites de los búferes receptores para prevenir desbordamientos de memoria, descartando cualquier trama corrupta o incoherente antes de consolidarla en el sistema.

3.3.2. Protección en la Capa Móvil

Dado que el teléfono inteligente almacena temporalmente información crítica de la explotación y credenciales de red, aplicamos las siguientes protecciones:

- **Custodia Segura de Credenciales:** Las claves de API para los servicios de Open-Meteo y los tokens de acceso para el servidor *Fog* no las incluimos como cadenas de texto plano en el código fuente. Las almacenamos cifradas en los contenedores de seguridad nativos provistos por el sistema operativo. *EncryptedSharedPreferences* en Android y *Keychain* en iOS.
- **Aislamiento en Tiempo de Ejecución:** El motor de inferencia ejecutado en la aplicación Flutter es beneficiado del entorno aislado de espacio de usuario del sistema operativo subyacente. Esto previene que otras aplicaciones instaladas en el dispositivo puedan acceder a la caché de telemetría de *IsarDB* o a la memoria RAM asignada para los modelos predictivos.

3.3.3. Seguridad Perimetral en el Servidor *Fog* Auto-Alojable

Dado que el servidor *backend Node.js* expone una API REST conectada a redes IP públicas o locales, aplicamos mecanismos rigurosos de endurecimiento:

- **Cifrado en Transmisión TLS:** Canalizamos obligatoriamente todo el tráfico de sincronización entre las aplicaciones móviles y el servidor sobre TLS 1.3, garantizando

⁴*Eavesdropping.*

⁵*Spoofing.*

la confidencialidad de la información y previniendo ataques de intermediario *Man-in-the-Middle*.

- **Autenticación Basada en Tokens y Claves Precompartidas:** Exigimos para el acceso a los *endpoints* de escritura y descarga de modelos la presentación de encabezados HTTP firmados mediante claves de API de alta entropía o tokens JWT.
- **Control de Tasa y Protección DoS:** Para mitigar ataques de denegación de servicio o saturación de peticiones desde dispositivos maliciosos, incorporamos en la API REST un limitador de tasa que restringe el número máximo de peticiones HTTP permitidas por dirección IP/Token dentro de una ventana temporal.
- **Aislamiento de Entorno mediante Docker:** Desplegamos el servidor *Node.js* y la base de datos SQLite en contenedores *Docker* aislados del sistema operativo anfitrión. Ejecutamos el proceso dentro del contenedor bajo un usuario sin privilegios de administración, eliminando el riesgo de escalada de privilegios o saltos de directorio en caso de que una vulnerabilidad sea explotada.

3.3.4. Soberanía y Privacidad de los Datos Agrícolas

Más allá de las barreras criptográficas, consideramos que la ventaja estructural más relevante de nuestra arquitectura híbrida radica en la preservación intransferible de la soberanía de los datos.

En los esquemas tradicionales basados al 100 % en la nube, las series temporales de la explotación, las estrategias de riego y el volumen de rendimiento agrícola se almacenan en servidores propiedad de terceros, exponiendo al agricultor a riesgos de monetización no autorizada de sus datos, fijación de precios especulativa o dependencias de suscripciones comerciales.

Nuestra solución aborda la privacidad mediante el paradigma *Privacy by Design*:

1. Procesamos los datos crudos de las sondas y los convertimos en veredictos predictivos en el propio hardware del agricultor.
2. Excluimos totalmente la nube comercial del *pipeline* crítico de decisión.
3. Realizamos la sincronización histórica exclusivamente contra nuestro servidor *Fog* auto-alojable, cuyo control físico y lógico pertenece al propio agricultor, la cooperativa o la comunidad de regantes.

De este modo, garantizamos que la información agrícola estratégica no abandone el perímetro digital de la explotación, asegurando una total independencia tecnológica y comercial.

CAPÍTULO 4

Desarrollo de la Aplicación Móvil

El desarrollo de la aplicación móvil conforma el componente lógico principal de la arquitectura en este TFM. Diseñamos el *software* siguiendo principios de modularidad. La aplicación opera como un nodo de procesamiento local en el *Edge*.

Empleamos el entorno multiplataforma *Flutter* y el lenguaje *Dart*. Este enfoque abstracta las rutinas nativas del dispositivo, como el acceso a la pila de comunicaciones BLE. Además, proporciona el rendimiento necesario para ejecutar los modelos de inferencia y gestionar las respuestas asíncronas de la red.

Estructuramos el proyecto basándonos en el *Clean Architecture* y el diseño guiado por el dominio o *Domain-Driven Design*. Dividimos el código fuente en tres capas lógicas para facilitar su mantenimiento y escalabilidad, como detalla la Figura 4.1:

- **Capa Central o Core:** Aloja la infraestructura base del sistema. Une la persistencia de datos local y la definición de modelos inmutables. También maneja la configuración de red y la inyección de dependencias. Para esto último utilizamos un gestor de rutinas global llamado *CliRoutines*.
- **Capa de Características o Features:** Encapsula la lógica de negocio específica y diferencial de la aplicación. Subdividimos esta capa en dominios delimitados. Incluye la orquestación BLE en el servicio *BleService*. También integra el motor de inferencia *InferenceBridge* y el consumo de variables climáticas.
- **Capa de Presentación o UI:** Agrupa la interfaz gráfica y la gestión de estado de las vistas. Implementa diseños adaptativos o *Responsive Design*. Estos diseños se modifican según la plataforma de ejecución. Las interfaces consumen de forma reactiva los flujos asíncronos emitidos por las capas inferiores.

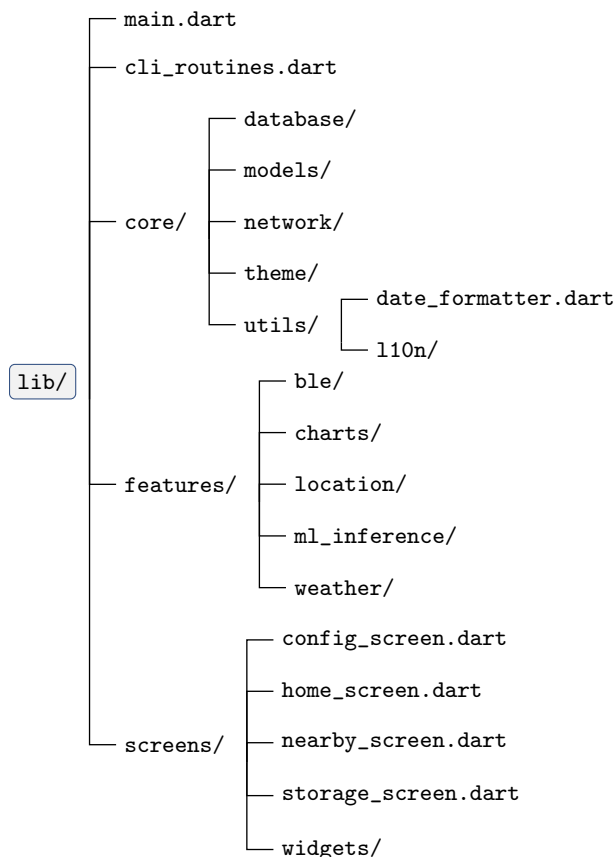


Figura 4.1: Estructura de archivos y módulos del *software* desarrollado para el nodo *Edge*.

4.1 Módulo Central

El módulo central provee los servicios de infraestructura a toda la aplicación. Aislar la base de datos y las peticiones de red asegura que las capas superiores operen sin acoplarse a librerías de terceros específicas.

4.1.1. Base de Datos

Las redes celulares en entornos rurales sufren desconexiones frecuentes. Por ello habilitamos el funcionamiento *offline-first* mediante el motor de base de datos *NoSQL IsarDB* [13]. Esta herramienta almacena la información localmente en el dispositivo con un consumo de recursos muy contenido.

Optimizamos la inserción de telemetría implementando diccionarios en memoria, como muestra el Código 4.1. Este diseño permite buscar y actualizar registros existentes con una complejidad constante $O(1)$. Así evitamos iterar repetidamente sobre listas extensas al filtrar lecturas duplicadas.

Añadimos rutinas de saneamiento temporal explícitas en el Código 4.2. Esta función elimina los registros cuyas marcas de tiempo indiquen fechas futuras. Esto previene fallos lógicos en los modelos de predicción causados por posibles desajustes del reloj interno de la estación IoT.

```

FUNCIÓN insertar_o_actualizar_telemetria
ENTRADAS: id_dispositivo, nuevos_valores, es_desde_nube
  
```

```

dispositivo = Buscar o crear dispositivo por id_dispositivo
historial = Obtener lista de registros históricos del
    dispositivo
hay_cambios = FALSO

// Construcción de mapa hash para búsquedas de complejidad O(1)
mapa_indices = NUEVO DICCIONARIO
PARA CADA registro EN historial HACER
    clave = Concatenar registro.tiempo, registro.profundidad y
        registro.tipo
    mapa_indices[clave] = posicion_actual
FIN PARA

// Procesamiento de los datos entrantes
PARA CADA nuevo_registro EN nuevos_valores HACER
    SI nuevo_registro.tiempo ES NULO ENTONCES CONTINUAR

    clave = Concatenar nuevo_registro.tiempo, nuevo_registro.
        profundidad y nuevo_registro.tipo
    indice = Buscar clave EN mapa_indices

    SI indice EXISTE ENTONCES
        SI historial[indice].valor ES DIFERENTE A
            nuevo_registro.valor ENTONCES
                historial[indice].valor = nuevo_registro.valor
                historial[indice].tipo = nuevo_registro.tipo
                historial[indice].puerto = nuevo_registro.puerto
                hay_cambios = VERDADERO
        FIN SI
    SINO
        Añadir nuevo_registro a historial
        mapa_indices[clave] = nueva_posicion_final
        hay_cambios = VERDADERO
    FIN SI
FIN PARA

// Aplicación de cambios en la persistencia
SI hay_cambios ES VERDADERO O es_desde_nube ES VERDADERO
    ENTONCES
        dispositivo.historial = historial
        dispositivo.fecha_actualizacion = Tiempo Actual
        dispositivo.sincronizado = es_desde_nube
        Guardar dispositivo en base de datos
    FIN SI

    RETORNAR hay_cambios
FIN FUNCIÓN

```

Código 4.1: Pseudocódigo del servicio de persistencia y telemetría

```

FUNCIÓN limpiar_datos_futuros_corruptos
    ENTRADAS: id_dispositivo

    dispositivo = Buscar dispositivo por id_dispositivo
    SI dispositivo NO EXISTE ENTONCES RETORNAR

    tiempo_actual = Tiempo Actual

```

```
historial_limpio = NUEVA LISTA

// Filtrado de anomalías temporales
PARA CADA registro EN dispositivo.historial HACER
    SI registro.tiempo ES MENOR O IGUAL A tiempo_actual
        ENTONCES
            Añadir registro a historial_limpio
        FIN SI
FIN PARA

// Detección de alteraciones y guardado
SI TAMAÑO DE historial_limpio ES DIFERENTE A TAMAÑO DE
dispositivo.historial ENTONCES
    dispositivo.historial = historial_limpio
    Guardar dispositivo en base de datos
FIN SI
FIN FUNCIÓN
```

Código 4.2: Pseudocódigo de la rutina de curación de datos.

4.1.2. Modelo de Datos

Definimos objetos de transferencia de datos o Data Transfer Objects (DTO) para estandarizar la información interna. Estas clases fuertemente tipadas previenen errores de acceso a memoria durante la ejecución.

Transformamos las tramas binarias codificadas bajo el estándar CBOR[5] de la conexión BLE en estructuras manejables en *Dart*. Aplicamos un proceso idéntico para procesar las respuestas en formato de texto JSON provenientes de la nube.

Toda entidad almacenada en la base de datos incluye un atributo booleano de sincronización. Este campo controla de manera directa qué registros permanecen pendientes de envío hacia el servidor. La Figura 4.2 muestra la relación directa entre el dispositivo y sus colecciones de valores.

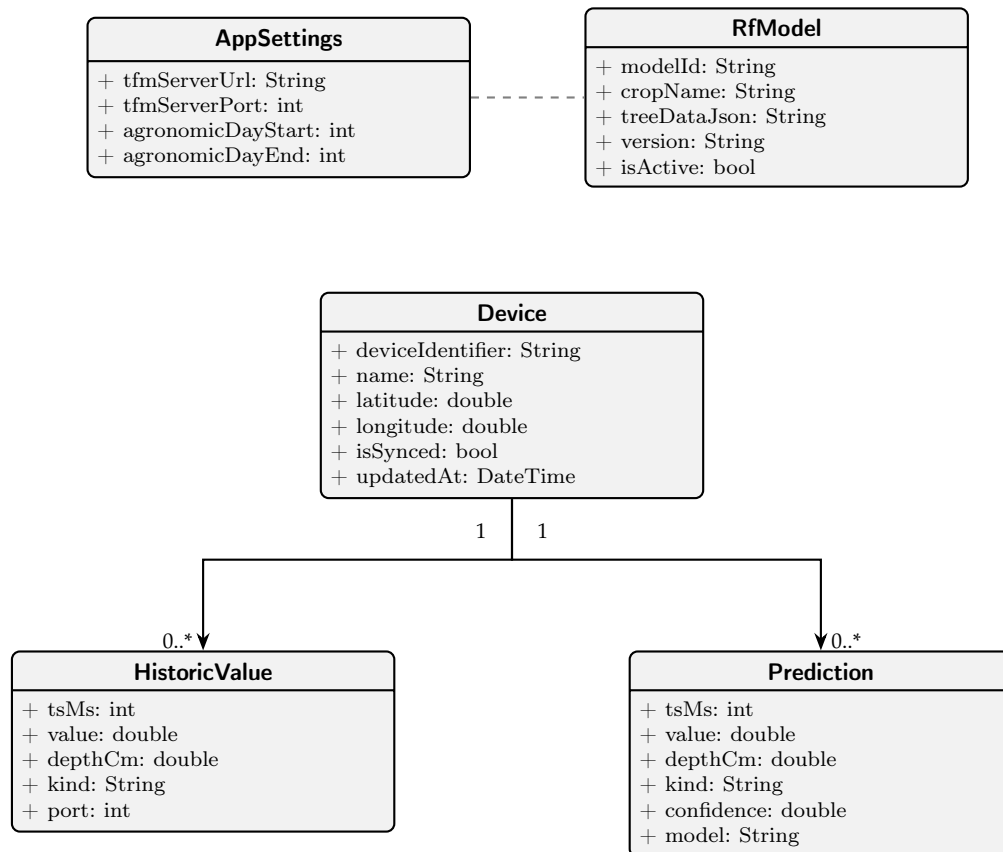


Figura 4.2: Diagrama de clases de las entidades principales del dominio para la persistencia local en *IsarDB*.

4.1.3. Red

Estructuramos la capa de red mediante clientes HTTP dedicados para desacoplar las responsabilidades. Por un lado, integramos un cliente para el consumo de la API meteorológica pública de Open-Meteo; por otro, centralizamos la interacción con el servidor de la plataforma en un cliente dedicado. Esto es visible en el Código 4.3.

En este último, configuramos tiempos límite fijos para abortar peticiones bloqueadas por mala cobertura e inyectamos cabeceras de autorización con *Bearer Tokens* para acceder a rutas protegidas. Utilizamos este mismo mecanismo para descargar los archivos de texto JSON que contienen la estructura y los pesos de los árboles de decisión.

```

FUNCIÓN ejecutar_peticion_con_reintentos
ENTRADAS: rutas_candidatas, token_autorizacion,
          tiempo_limite_segundos

  // Construcción e inyección centralizada de cabeceras de
  // autorización
  cabeceras = NUEVO DICCIONARIO
  cabeceras["Content-Type"] = "application/json"
  SI token_autorizacion NO ES NULO Y token_autorizacion NO ESTÁ
  VACÍO ENTONCES
    cabeceras["Authorization"] = "Bearer " + token_autorizacion
  FIN SI

  // Política de reintento y tolerancia a fallos sobre endpoints
  // alternativos
  PARA CADA url_endpoint EN rutas_candidatas HACER

```

```

    INTENTAR
        // Ejecución de la petición abortando si se supera el
        // tiempo límite por baja cobertura
        respuesta = Petición GET HTTP a url_endpoint CON
        cabeceras Y TIEMPO_LÍMITE tiempo_limite_segundos
        SI respuesta.codigo_estado ES IGUAL A 200 ENTONCES
            RETORNAR respuesta.cuerpo_texto // Retorna datos o
            JSON con pesos del modelo
        FIN SI
        CAPTURAR ERROR_CONEXION_O_TIMEOUT
        // Si ocurre un error de red o timeout, se continúa
        // con el siguiente intento/ruta
        CONTINUAR
    FIN INTENTAR
FIN PARA

LANZAR EXCEPCIÓN "Error al completar la petición HTTP tras
agotar todas las rutas y reintentos"
FIN FUNCIÓN

```

Código 4.3: Pseudocódigo del cliente HTTP centralizado con inyección de cabeceras, tiempos límite y política de reintentos.

4.1.4. Utilidades y Diseño

Agrupamos las variables gráficas y los textos estáticos en módulos de utilidades. Establecemos una paleta de colores de tema oscuro para mejorar el contraste bajo la luz solar directa.

Aprovechamos los componentes de estado de *Flutter* para redibujar la pantalla. La interfaz se actualiza únicamente cuando detecta cambios en las variables que observa. Este diseño reactivo evita recargas innecesarias y mantiene la fluidez de navegación.

4.2 Módulo de Características

Este módulo concentra la lógica operativa principal de la aplicación. Conecta la recolección de datos procedentes de los sensores con la emisión de recomendaciones agronómicas.

4.2.1. Bluetooth

Utilizamos la librería externa *FlutterBluePlus* para gestionar la conexión con el microcontrolador. Filtramos el escaneo inicial utilizando el identificador de servicio específico de nuestro *hardware* para ignorar dispositivos ajenos.

Implementamos un paso de autenticación obligatorio en la clase *PicoHandshakeModule*. El proceso consiste en firmar un fragmento aleatorio “*nonce*” mediante el algoritmo de hash SHA-256 utilizando la contraseña del usuario como llave. Una vez establecida la confianza, el ensamblador agrupa los fragmentos de datos CBOR entrantes y los emite al resto del sistema mediante *streams*.

4.2.2. Machine Learning

El submódulo de inferencia calcula la necesidad de riego directamente en el procesador del teléfono móvil. Consultamos el registro local en primera instancia para extraer la humedad del suelo reciente. Calculamos una aproximación matemática de la radiación solar basada en la latitud y fecha si el dispositivo carece de conexión a la API meteorológica.

Desarrollamos la clase *DynamicRandomForest* para realizar la evaluación final. Este componente interpreta un árbol de decisión exportado a formato JSON, resultando en un compensación que permite adaptar el funcionamiento de un árbol de decisiones (Figura 4.3) de forma dinámica. Cruzamos las características extraídas con las reglas dictadas por el archivo descargado para emitir una recomendación booleana de riego.

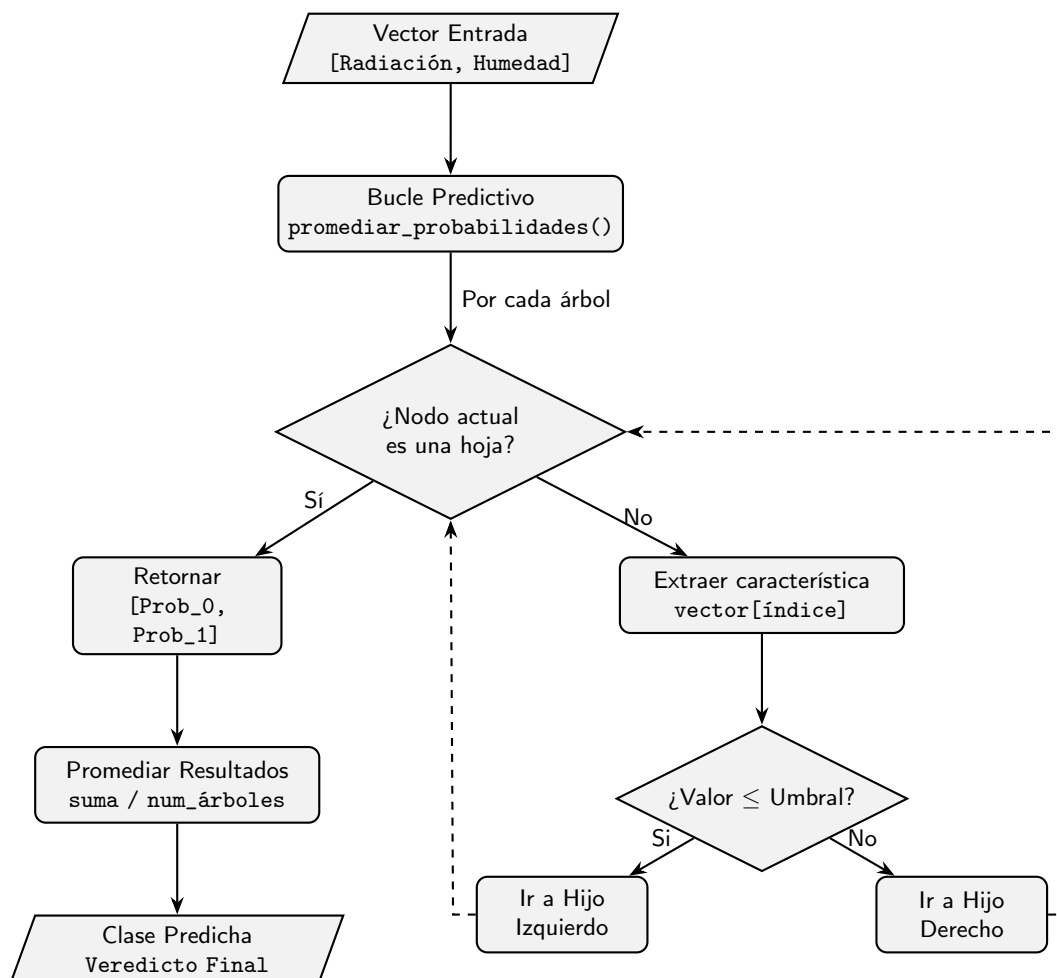


Figura 4.3: Diagrama de flujo del recorrido de árboles y clasificación probabilística en la clase *DynamicRandomForest*.

4.3 Módulo de Interfaz de Usuario y Gestión de Estado

La capa de presentación de la aplicación móvil constituye la interfaz de control y supervisión para el usuario agrícola. Más allá del diseño puramente visual, el reto arquitectónico en una aplicación *Edge* radica en mantener una interfaz fluida a 60 cuadros por segundo mientras se ejecutan tareas intensivas de red, criptografía e inferencia ma-

temática en el mismo dispositivo. Para resolver este cuello de botella, estructuramos una gestión de estado altamente reactiva combinada con un paralelismo estricto.

Delegamos cualquier carga computacional densa, como el desempaquetado de las tramas binarias CBOR, el cálculo del *hash* SHA-256 para el *Handshake* BLE, o la evaluación recursiva de los árboles de decisión JSON, a hilos de ejecución secundarios independientes, denominados *Isolates* en el ecosistema Dart. A diferencia de las promesas asíncronas tradicionales, los *Isolates* operan en su propio espacio de memoria y se comunican con el hilo principal a través del paso de mensajes. Esto garantiza que la interfaz gráfica nunca se congele durante el postprocesado de telemetría masiva.

Para la gestión del estado reactivo de la UI, implementamos un patrón basado en el consumo de flujos de datos o *Streams* asíncronos. La interfaz se suscribe a los repositorios subyacentes y se redibuja automáticamente solo cuando se emite un nuevo evento, por ejemplo, cuando finaliza una inferencia o cuando el RSSI del BLE cambia. Este diseño reactivo evita recargas innecesarias y mantiene la fluidez de navegación en condiciones operativas exigentes.

Además, el sistema de renderizado evalúa dinámicamente las restricciones espaciales del lienzo lógico del dispositivo, estrategia que sigue el patrón *Responsive*:

- **Disposición Móvil:** En pantallas de ancho reducido, la navegación se encapsula en una barra inferior *BottomNavigationBar*.
- **Disposición Tablet o Landscape:** En pantallas extendidas, el *layout* conmuta automáticamente hacia un riel lateral de navegación permanente *NavigationRail*, maximizando el espacio de visualización de gráficas y telemetría sin requerir intervención del usuario.

Esta adaptabilidad espacial se ilustra esquemáticamente en la Figura 4.4, garantizando la continuidad operativa sin importar el dispositivo utilizado.

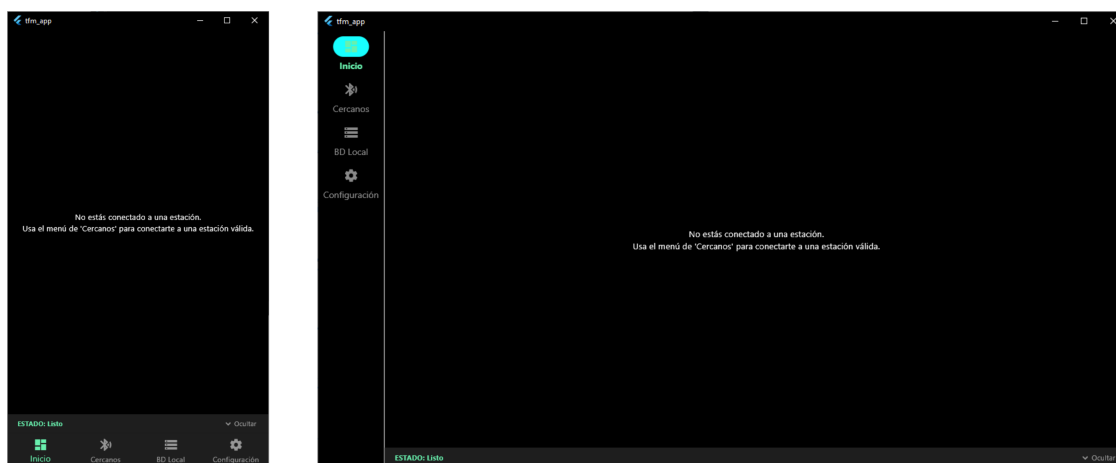


Figura 4.4: Interfaz de usuario según la resolución del dispositivo. Barra inferior vs. Riel lateral.

4.3.1. Menú Principal o *Dashboard*

El Menú Principal o pantalla de inicio actúa como el centro de mando operativo del sistema, agregando las variables microclimáticas, el estado telemétrico y las predicciones analíticas en un único panel coherente, tal como se aprecia en la Figura 4.5. Sus componentes principales se estructuran de la siguiente forma:

- **Indicador de Estado de Enlace BLE:** Presenta en tiempo real el estado de sincronización con el nodo de campo: *Desconectado*, *Escaneando*, *Conectado*, junto con la dirección identificadora del dispositivo enlazado y la hora exacta del último volcado telemétrico.
- **Tarjeta de Inferencia y Recomendación Agronómica Local:** Destaca de forma prominente el veredicto emitido por el ecosistema predictivo en el *Edge* (LSTM y *Random Forest*). Expone de manera determinista:
 - La decisión operativa: Riego Requerido o Riego Evitable.
 - El resultado del modelo predictivo utilizado para la generación del dictamen.
 - La estimación cuantitativa de la humedad prevista a 24 horas (T_{24}).
- **Salida de Consola:** Informa de los resultados de cada una de las acciones de transacción o inferencia realizadas en el sistema de predicción, ofreciendo una descarga directa en formato JSON o una copia al portapapeles del dispositivo.

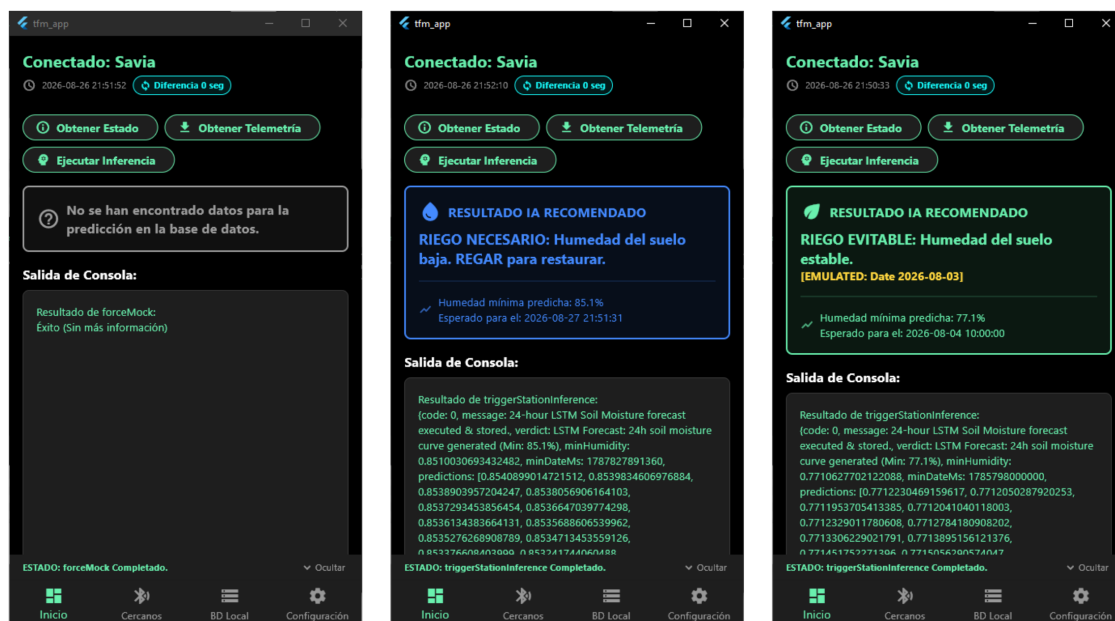


Figura 4.5: Vista del Menú Principal detallando el indicador de enlace, la tarjeta de recomendación agronómica y la salida de consola. Esta figura muestra tres casos: 1. Conexión con estación IoT sin transacciones iniciadas. 2. Estimación del riego según el tiempo actual, en este caso “riego necesario”. 3. Estimación del riego según la última información guardada en la estación, por lo que la lógica del sistema emula la predicción bajo dichas condiciones, en este caso “riego evitable” para el 4/8/2026.

4.3.2. Menú de Conexión Bluetooth

Esta interfaz gestiona el descubrimiento, la auditoría de cobertura de radiofrecuencia y el emparejamiento con los nodos IoT distribuidos en las parcelas (Figura 4.6). El flujo operativo se desglosa en los siguientes elementos itemizados:

- **Descubrimiento Dinámico de Nodos en Proximidad:** Ejecuta escaneos periódicos en segundo plano, listando los dispositivos que anuncian servicios GATT compatibles. Cada entrada muestra la denominación comercial de la estación y su dirección MAC única.

- **Auditoría de Potencia de Señal:** Visualiza la intensidad del enlace de radio en decibelios-miliovatio. Permite al operador evaluar físicamente la calidad del enlace de radio antes de iniciar la transferencia de datos.
- **Autenticación y Emparejamiento:** Incluye un campo de entrada con enmascaramiento de caracteres para introducir el código PIN de seguridad precompartido ¹, impidiendo enlaces no autorizados de terceros sobre las estaciones de campo.

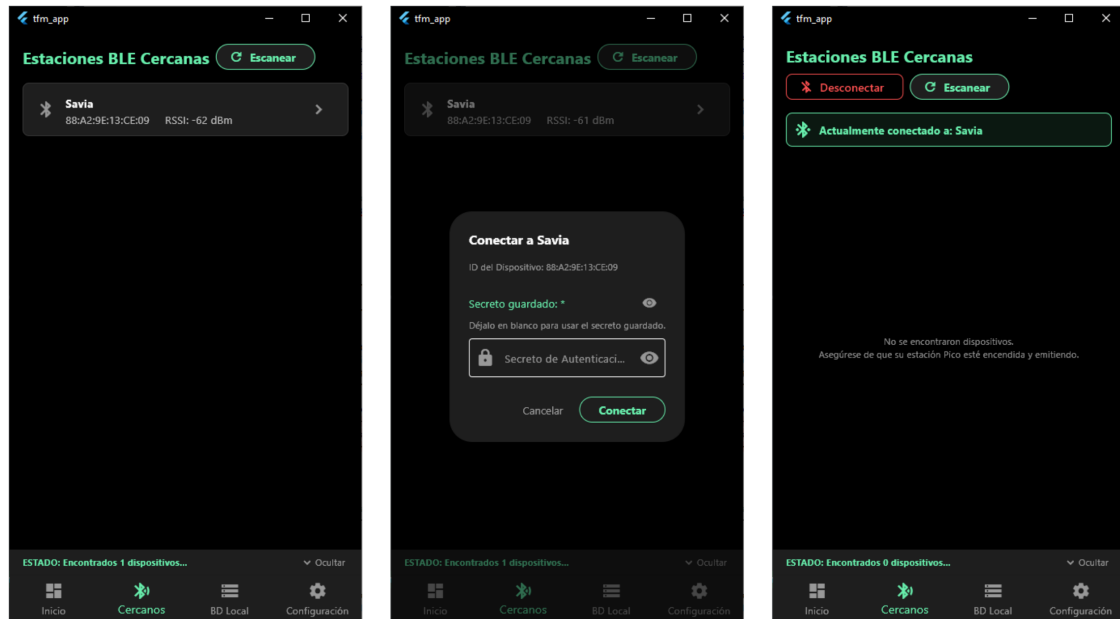


Figura 4.6: Interfaz del Menú de Conexión Bluetooth con listado de nodos descubiertos y niveles de RSSI, formulario de emparejamiento, y conexión exitosa.

4.3.3. Menú de Almacenamiento y Persistencia

La vista de almacenamiento actúa como un panel de control unificado de estaciones que permite supervisar, auditar y sincronizar las entidades tanto en el almacenamiento local como en la infraestructura central *Fog*, ilustrado en la Figura 4.7. Sus capacidades operativas las desglosamos en los siguientes módulos:

- **Panel Superior de Control y Diagnóstico de Red:**
 - **Estado de Conectividad Fog:** Renderiza el estado dinámico del enlace telemático con el servidor central: *CONNECTED*, *UNREACHABLE*, *TESTING* . . . o *ERROR* e indica el conteo de dispositivos locales pendientes de sincronización.
 - **Prueba de API Fog:** Botón dedicado para ejecutar un test de conectividad y latencia contra el extremo remoto.
 - **Sincronización Bidireccional:** Orquesta un flujo asíncrono que vuelca los registros locales modificados, descubre nuevos nodos en el servidor máster y descarga la telemetría histórica mediante consultas HTTP.
 - **Purga del Repositorio Local:** Acción destructiva que despliega un diálogo de confirmación para el borrado seguro de la base de datos embebida.

¹Passkey.

- **Listado de Estaciones Unificadas:** Agrega en una lista interactiva las estaciones locales y remotas. Cada tarjeta presenta:
 - El nombre de la estación y su identificador físico *ID*.
 - Etiquetas de presencia: Identificadores visuales que indican si el nodo está guardado en la base de datos embebida o si se encuentra registrado en el servidor central.
 - Metadatos de posición y volumen telemétrico: Coordenadas geográficas de latitud y longitud, acompañadas del volumen total de registros de telemetría custodiados localmente.
- **Submenú Operativo por Estación y Visor Cartográfico:** Al seleccionar una estación, la tarjeta despliega un panel extensible con opciones de ejecución:
 - **Inferencia Local sobre DB:** Dispara el motor analítico sobre las series temporales de la estación en Isar DB.
 - **Emulación Fog en Memoria:** Ejecuta una simulación en memoria del algoritmo de predicción alojado en el servidor *Fog*.
 - **Visor Cartográfico Interactivo:** Abre una ventana emergente que integra un mapa de *OpenStreetMap* para situar espacialmente las coordenadas del sensor.
 - **Tarjeta de Resultados de IA:** Muestra de forma detallada el dictamen emitido: RIEGO NECESARIO / EVITABLE, la humedad predicha a 24h, la radiación solar acumulada y la validación con la ventana agronómica diaria.

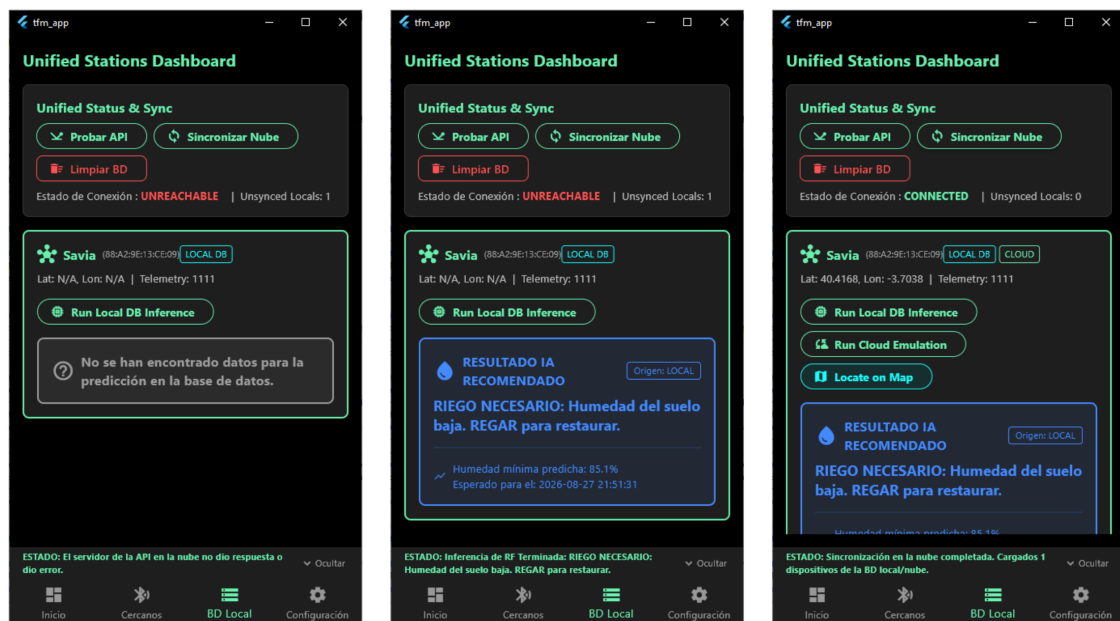


Figura 4.7: Pantalla del Menú de Almacenamiento con panel de diagnóstico de API *Fog*, listado unificado de estaciones con etiquetas LOCAL DB / FOG-SERVER y submenú de inferencia.

4.3.4. Menú de Configuración Paramétrica

El Menú de Configuración expone los parámetros estáticos del sistema, permitiendo la adaptación de la infraestructura de red, la lógica de inferencia, y los modelos de aprendizaje automático sin necesidad de recompilar el código fuente, ilustrado en la Figura 4.8. La interfaz organiza sus controles en cuatro tarjetas especializadas y un módulo flotante de gestión de modelos:

- **Cabecera y Persistencia Global:** Incorpora un botón de aplicación inmediata *Apply & Save*, que consolida los ajustes en la base de datos embebida, actualiza los *end-points* de red en tiempo real en la memoria de ejecución y propaga las claves de autenticación hacia los servicios activos.
- **Entorno y Sistema:**
 - **Reloj del Sistema en Tiempo Real:** Visualiza la fecha, la hora local con precisión de segundos y la marca temporal UNIX en milisegundos.
 - **Gestión de Geolocalización del Cultivo:** Expone las coordenadas geográficas vigentes en latitud y longitud, y su modo de adquisición sea GPS o Manual.
 - **Selector de Modo de Ubicación:** Permite optar entre la captura automática mediante el sensor GPS del terminal o la selección manual interactiva mediante un mapa vectorial creado utilizando *FlutterMap* y *OpenStreetMap*, donde el usuario posiciona un marcador táctil sobre la parcela.
- **Red y Servicios Cloud:**
 - **Diagnóstico de la API Meteorológica *Open-Meteo*:** Muestra el estado de la comunicación telemática: *Meteo OK*, *Error HTTP* o *Offline* con un botón para ejecutar un diagnóstico de red en tiempo real.
 - **Endpoint del Servidor Central *Fog*:** Muestra el esquema de red, la dirección IP/dominio y el puerto configurados. Por ejemplo: `http://192.168.1.50:3000`, junto con el resultado cuantitativo de latencia. Incluye diálogos dedicados para la edición y prueba de conectividad.
 - **Clave de Autenticación de API:** Administra la clave secreta precompañada con la infraestructura central mediante un campo de texto con visibilidad conmutable.
- **Ventana Agronómica:**
 - **Periodo Recomendado de Riego:** Define el intervalo horario donde suele tomar parte la actividad de riego. Por defecto, 10:00h – 18:00h.
 - **Periodo de Inferencia y Predicción:** Establece la ventana horaria en la que el sistema evalúa la necesidad hídrica para el próximo periodo de riego. Por defecto, 19:00h – 9:00h.
 - **Controles de Desplazamiento y Verificación:** Habilita botones de ajuste fino (± 1 hora) sobre ambos rangos, imponiendo límites de seguridad de ± 3 horas respecto a las constantes por defecto e informando de posibles discrepancias mediante avisos de advertencia.
- **Tarjeta y Catálogo de Modelos *Machine Learning*:** Expone el modelo *Random Forest* activo, compilado por defecto o personalizado por cultivo, y despliega una hoja modal deslizable que permite:
 - Consultar el catálogo remoto alojado en el servidor máster con especificación del tipo de cultivo, versión y tamaño en kilobytes.
 - Descargar dinámicamente modelos JSON en la memoria del dispositivo.
 - Conmutar el modelo activo o eliminar modelos obsoletos de la base de datos local.

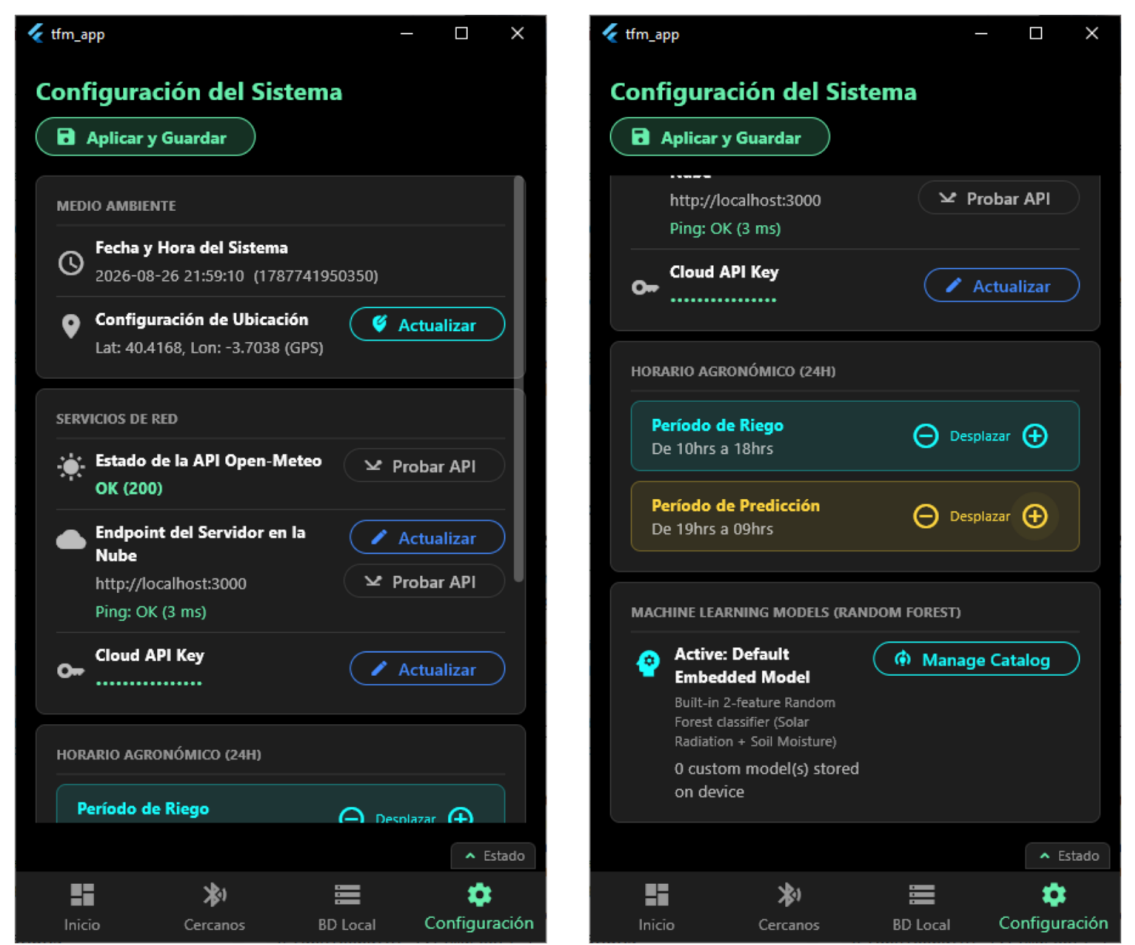


Figura 4.8: Menú de Configuración mostrando las tarjetas de entorno, diagnóstico de red, ventana agronómica y el catálogo de modelos *Random Forest*.

CAPÍTULO 5

Integración de *Hardware* y Flujo de Telemetría

Este capítulo detalla la vinculación directa entre nuestra arquitectura móvil en el *Edge* y la placa física de instrumentación en el *Deep Edge*. Si bien la instrumentación electrónica y el manejo de los sensores físicos fueron desarrollados en colaboración con [4], una parte del diseño, la implementación de la máquina de estados para el reensamblado BLE, la adaptación de los protocolos de serialización y la orquestación del flujo telemétrico constituyen aportaciones centrales de este TFM.

Desglosamos a continuación las decisiones estructurales de diseño y los problemas de ingeniería que resolvimos para garantizar una comunicación eficiente y estable en el borde. Analizamos el desarrollo de bajo nivel y la implementación de los protocolos inalámbricos. El código fuente de la estación IoT emplea los lenguajes C y C++.

La estación opera sobre los microcontroladores RP2040 y RP2350. Aunque las pruebas iniciales de instrumentación contemplaron arquitecturas base como el RP2040, el despliegue final y la evaluación de esta arquitectura operan exclusivamente sobre la plataforma Raspberry Pi Pico 2 W (RP2350). Estos chips imponen restricciones severas de memoria y procesamiento. Se utilizan los conversores analógico-digitales internos, así se adquieren las lecturas de los sensores en campo.

La autonomía del dispositivo se ha maximizado mediante un ciclo estricto de suspensión profunda. El microcontrolador desactiva completamente el procesador de radio inalámbrico durante los periodos inactivos. El sistema despierta temporalmente. En este breve periodo mide variables o procesa rutinas de inteligencia artificial.

5.1 Codesarrollo de Protocolos y *Firmware*

Aportamos contribuciones fundamentales al repositorio del *firmware*. El objetivo es alinear el comportamiento del dispositivo con nuestra aplicación en el *Edge*. La Tabla 5.1 muestra un resumen.

Modificamos la capa de serialización y la gestión de las conexiones entrantes.

Definimos una frontera clara de responsabilidades con nuestro compañero de desarrollo [4]. El *firmware* lee los puertos físicos y mantiene el reloj en tiempo real. Nuestra aplicación móvil actúa como pasarela táctica. También provee el contexto externo.

Mejoramos el decodificador binario integrado en el microcontrolador. Añadimos soporte nativo para leer números de coma flotante de precisión media. Los clientes móviles


```

        FIN SI
        SI modo_transaccion ES IGUAL A MODO_ACTIVO ENTONCES
            RETORNAR 0
        FIN SI
        SINO SI modo_transaccion ES IGUAL A MODO_CANCELAR ENTONCES
            weather_len = 0
            RETORNAR 0
        FIN SI

        SI modo_transaccion ES IGUAL A MODO_EJECUTAR O
        modo_transaccion ES IGUAL A MODO_NINGUNO ENTONCES
            (temp_pasadas, temp_futuras, n_pas, n_fut) =
                DECODIFICAR_CBOR_CLIMA(weather_buf, weather_len)
            ACTUALIZAR_CACHE_CLIMA(temp_pasadas, n_pas,
                temp_futuras, n_fut, OBTENER_TIEMPO_RTC())
            weather_len = 0
        FIN SI

        SINO SI handle_caracteristica ES IGUAL A H_DATA_REQUEST Y
        TIPO_DE_OPERACION(buffer_datos) ES IGUAL A "infer" ENTONCES
            inferencia_pendiente = VERDADERO
        FIN SI
    FIN FUNCIÓN

FUNCIÓN bucle_principal_superloop
    ENTRADAS: ninguna

    MIENTRAS VERDADERO HACER
        EJECUTAR_TAREAS_RADIO_BLE()

        SI inferencia_pendiente ES IGUAL A VERDADERO ENTONCES
            inferencia_pendiente = FALSO

            BLOQUEAR_INTERRUPCIONES_RADIO()
            EJECUTAR_INFERENCIA_ON_DEVICE_LSTM(OBTENER_TIEMPO_RTC()
            )

            DESBLOQUEAR_INTERRUPCIONES_RADIO()
        FIN SI

        ENTRAR_EN_MODO_BAJO_CONSUMO_HASTA_SIGUIENTE_EVENTO()
    FIN MIENTRAS
FIN FUNCIÓN

```

Código 5.1: Pseudocódigo del mecanismo de bloqueo asíncrono y reensamblado de paquetes largos en memoria estática.

5.2 Protocolo de comunicación BLE

Implementamos la comunicación BLE utilizando la librería nativa *FlutterBluePlus*. Desarrollamos una interfaz de programación intermedia para orquestar la transferencia de datos entre el dispositivo móvil y el microcontrolador.

5.2.1. Perfil de Atributos Genérico GATT

Estructuramos la capa de comunicaciones inalámbricas del nodo periférico sobre el perfil GATT del estándar BLE. Junto con [4], diseñamos un servicio primario personalizado que agrupa ocho características dedicadas para gobernar de forma determinista el control de acceso, la telemetría periódica, la inyección climática y la transferencia masiva de registros.

Para mantener la coherencia en la pila de red, asignamos identificadores universales únicos de 128 bits bajo un espacio de direcciones homogéneo. El servicio raíz centraliza todas las operaciones de la estación agroclimática bajo la base común $5a71a000 - 0000 - 0000 - 0000 - 0000000000XY$, mientras que las características adoptan sufijos según su dominio funcional: desde $0x10$ a $0x21$. En esta topología, destinamos canales específicos para la lectura de estado ($0x10$), la sincronización del reloj interno ($0x11$), la ingesta de previsiones meteorológicas ($0x12$) y la autenticación por desafío ($0x14$). Paralelamente, desacoplamos la transferencia de datos masivos en dos cauces complementarios: un canal receptor de peticiones operativas ($0x20$) y un canal emisor de notificaciones asíncronas troceadas ($0x21$).

Los canales $0x13$ y $0x15$ pertenecen al flujo de configuración de la estación IoT [4], por lo que no son utilizados en este TFM.

En la Tabla 5.2 sintetizamos la totalidad de los canales de comunicación GATT definidos para el nodo periférico junto con sus permisos de acceso y funciones asociadas.

UUID de 128 bits	Identificador	Propiedad	Función y Carga Útil
Servicio Primario <i>Savia</i> IoT			
$5a71a000 - . . 01$	SAVIA_SVC	—	Agrupación raíz de control telemétrico e inferencia.
Características GATT			
$5a71a000 - . . 10$	CHR_STATUS	<i>Read</i>	Lectura de telemetría instantánea y estado de batería en mapa CBOR.
$5a71a000 - . . 11$	CHR_TIME_SYNC	<i>Write</i>	Sincronización del reloj en tiempo real mediante marca temporal UNIX en milisegundos.
$5a71a000 - . . 12$	CHR_WEATHER	<i>Write</i>	Inyección manual de vectores térmicos pasados y pronosticados en formato CBOR de 32 bits.
$5a71a000 - . . 13$	CHR_CONFIG	<i>Read / Write</i>	Configuración de parámetros operativos y selección de modo de inferencia local o remoto.
$5a71a000 - . . 14$	CHR_AUTH	<i>Read / Write</i>	Intercambio de desafío criptográfico <i>nonce</i> , firma SHA-256 y provisión de contraseña.
$5a71a000 - . . 15$	CHR_PINMAP	<i>Read</i>	Consulta del esquema de patillaje y sensores físicos conectados a la placa.
$5a71a000 - . . 20$	CHR_DATA_REQ	<i>Write</i>	Solicitud de series temporales históricas, purga de memoria y disparo de inferencia.
$5a71a000 - . . 21$	CHR_DATA_RESP	<i>Notify</i>	Emisión asíncrona de tramas masivas fragmentadas mediante bloques binarios consecutivos.

Tabla 5.2: Mapa de UUIDs para el servicio primario y las características GATT del nodo de campo.

Fundamentamos esta estructura modular en dos decisiones orientadas a la eficiencia:

- **Filtrado pasivo en el descubrimiento:** Centralizar las funciones bajo un único identificador de servicio permite a la aplicación móvil aplicar un filtrado por *hardware* en el controlador de radio. Al restringir el escaneo exclusivamente a las tramas de anuncio que contengan dicho UUID y limitar la ventana activa a quince segundos, minimizamos el ciclo de trabajo del receptor, evitamos procesar tráfico electromagnético de dispositivos ajenos y aceleramos el establecimiento de la sesión.
- **Desacoplamiento asíncrono del flujo de datos:** Separar el canal de comandos del canal de respuesta (0x20 y 0x21) evita colapsar el hilo de control. La aplicación móvil emite órdenes puntuales de consulta sin bloquear la pila de transporte, mientras que el microcontrolador transmite las series temporales fragmentadas en lotes mediante notificaciones asíncronas no bloqueantes.

5.2.2. Estructuración del *Payload*

Descartamos el uso de texto plano para minimizar la sobrecarga en la transmisión. Empaquetamos todos los mensajes mediante el estándar binario CBOR. El *buffer* de recepción del microcontrolador impone un límite estricto de 512 *bytes*.

La biblioteca estándar de *Dart* comprime valores individuales a 16 *bits*. Por ello, adaptamos el *firmware* para admitir el marcador hexadecimal 0xF9. Sin embargo, dicha biblioteca utiliza coma flotante de 64 *bits* para arreglos numéricos extensos. Al procesar las 72 muestras de la previsión meteorológica, el paquete supera los 600 *bytes*. Esto desborda la memoria del nodo.

El entorno de *Dart* carece de funciones nativas para serializar flujos a 16 *bits*. Por esta razón, desarrollamos una rutina de construcción binaria manual a bajo nivel. Forzamos la escritura en precisión simple de 32 *bits* bajo el estándar IEEE 754. Cada número incorpora el marcador hexadecimal 0xFA. El pseudocódigo correspondiente está recogido en Código 5.2.

Con este método reducimos la carga útil final a 412 *bytes*. Este volumen garantiza la transferencia atómica de las 48 horas pasadas y las 24 horas pronosticadas. El mensaje completo entra en una única transmisión sin saturar el microcontrolador.

```

FUNCIÓN ConstruirPaqueteClima(T_pas, T_fut)
  B = NUEVA LISTA DE BYTES
  B.AñadirByte(0xA3) // Mapa raíz con 3
    elementos
  B.AñadirBytes([0x61, 0x76, 0x01]) // Versión: "v": 1
  B.AñadirBytes([0x62, 0x6F, 0x70, 0x63, 0x75, 0x70, 0x64]) //
    Operación: "op": "upd"
  B.AñadirBytes([0x64, 0x64, 0x61, 0x74, 0x61, 0xA2]) // Clave "
    data" (submapa de 2 elementos)

  clavePas = CodificarUTF8("past_ta_hourly")
  B.AñadirByte(0x60 | Longitud(clavePas))
  B.AñadirBytes(clavePas)
  B.AñadirBytes([0x98, 0x30]) // Arreglo de 48 elementos

  PARA CADA valor t EN T_pas HACER
    B.AñadirByte(0xFA) // Marcador CBOR float32
    bytesFloat = ConvertirA_IEEE754_Float32_BigEndian(t)
    B.AñadirBytes(bytesFloat)

```

```

FIN

claveFut = CodificarUTF8("future_ta_hourly")
B.AñadirByte(0x60 | Longitud(claveFut))
B.AñadirBytes(claveFut)
B.AñadirBytes([0x98, 0x18])           // Arreglo de 24 elementos

PARA CADA valor t EN T_fut HACER
    B.AñadirByte(0xFA)                 // Marcador CBOR float32
    bytesFloat = ConvertirA_IEEE754_Float32_BigEndian(t)
    B.AñadirBytes(bytesFloat)
FIN

RETORNAR B.ObtenerBytes()
FIN

```

Código 5.2: Construcción binaria manual del paquete CBOR para inyección meteorológica

El algoritmo recorre secuencialmente cada lectura térmica. Transforma cada valor de coma flotante en cuatro *bytes*. Aplica el orden de *byte* más significativo primero. Ensamblamos la trama directamente en memoria para impedir fallos en el nodo empotrado.

5.2.3. Estabilidad de Conexión y Manejo de Errores

Diseñamos rutinas específicas para garantizar la estabilidad del enlace en campo abierto. El cliente móvil negocia activamente una unidad máxima de transferencia o *MTU* de quinientos doce *bytes* al conectarse. La descarga de telemetría histórica excede la capacidad de un único mensaje de radio. El microcontrolador fragmenta la respuesta en múltiples paquetes indexados. Desarrollamos la clase *BleChunkAssembler* en la aplicación móvil para procesar y reconstruir estas transmisiones segmentadas.

Implementamos un módulo de autenticación criptográfica basado en el algoritmo *hash* SHA-256. El cliente lee un número aleatorio desde el sensor y lo firma digitalmente utilizando una contraseña compartida. El dispositivo valida esta firma para desbloquear el acceso a la telemetría.

Diseñamos una máquina de estados finitos para orquestar el proceso de ensamblado de segmentos, ilustrado en la Figura 5.1. La máquina parte de un estado de reposo a la espera de tráfico telemétrico. Al detectar un paquete válido, extrae su índice de secuencia y lo almacena en un diccionario temporal en memoria. Si la trama resulta corrupta, el sistema la descarta y retorna al estado de escucha. El algoritmo verifica dinámicamente la presencia de todos los fragmentos. Tras confirmar la ausencia de huecos mediante una bandera de fin de archivo, concatena los bloques validados. Finalmente, el sistema decodifica el objeto íntegro para su almacenamiento.

Incorporamos una protección adicional frente a transmisiones interrumpidas. Si el ensamblador recibe un fragmento con índice cero cuando conserva datos de una descarga previa incompleta, la rutina purga automáticamente el búfer antiguo. Esta regla previene la mezcla de telemetrías pertenecientes a diferentes sesiones de lectura.

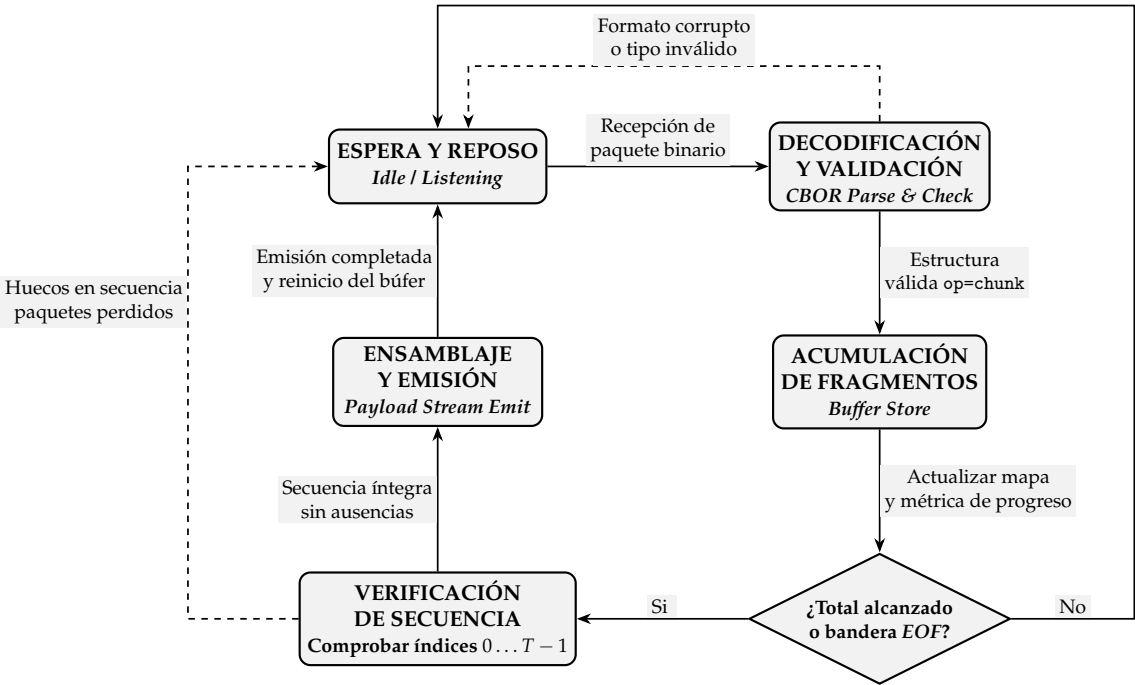


Figura 5.1: Diagrama de estados del ensamblador de fragmentos binarios BLE en la aplicación móvil.

CAPÍTULO 6

Almacenamiento Centralizado y Servidor Auto-Alojable

Este capítulo detalla la construcción del servidor *backend* o *Fog*. Explicamos las decisiones de diseño para lograr un sistema auto-alojable. Abordamos la arquitectura y el esquema de la base de datos. Detallamos también la interfaz de comunicación y la sincronización con la aplicación móvil.

Optamos por construir un servidor *self-hosted* en lugar de utilizar plataformas comerciales cerradas. Las soluciones privativas generan una dependencia tecnológica y económica importante. Esta dependencia resulta perjudicial para los agricultores independientes a largo plazo. Nuestra solución prioriza la soberanía de los datos agrícolas. Los agricultores mantienen el control total sobre su información telemétrica. Esto fomenta la privacidad y evita el bloqueo del proveedor.

Diseñamos el sistema para reducir los Operating Expenses (OPEX) drásticamente. El alojamiento en servidores virtuales de bajo coste minimiza los gastos mensuales. Un agricultor puede mantener la infraestructura con una inversión económica mínima. Buscamos también fomentar la interoperabilidad comunitaria. El servidor expone interfaces estandarizadas y abiertas. Diferentes actores logísticos pueden integrarse sin barreras tecnológicas restrictivas.

6.1 Arquitectura del servidor y *stack* tecnológico

La Figura 6.1 enseña un resumen general de la arquitectura. A continuación, la estaremos desglosando punto por punto.

Elegimos *Node.js* como entorno principal de ejecución. Esta tecnología ofrece un excelente rendimiento en operaciones de red asíncronas. Resulta ideal para procesar peticiones concurrentes de múltiples dispositivos remotos.

Integramos un servicio secundario escrito en lenguaje *Python*. Este servicio convierte los modelos estadísticos complejos a un formato ligero en JSON. El servidor principal lanza este subproceso automáticamente durante su arranque.

Empaquetamos el servidor utilizando contenedores *Docker*. La contenerización asegura un entorno de ejecución predecible y reproducible en cualquier máquina. Eliminamos así los problemas de dependencias en diferentes sistemas operativos.

Mapeamos volúmenes persistentes para salvaguardar la base de datos y los modelos estáticos. El contenedor expone sus puertos directamente para recibir las peticiones. Prescindimos de servidores web intermedios pesados para optimizar los recursos.

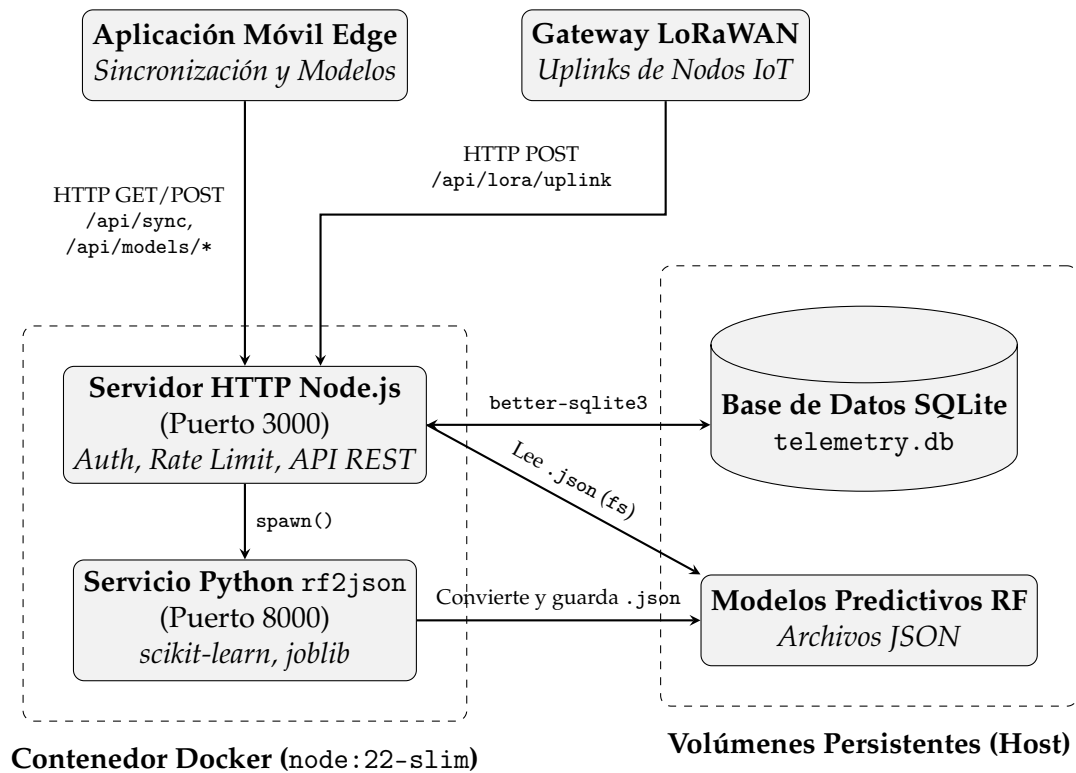


Figura 6.1: Arquitectura del servidor y contenedores Docker.

6.2 Esquema de base de datos

Implementamos *SQLite* como motor de base de datos relacional. Esta herramienta simplifica enormemente la gestión del almacenamiento local. No requiere procesos independientes ni configuraciones complejas en el servidor. Utilizamos sentencias precompiladas para optimizar el rendimiento de lectura y escritura. Esto permite inserciones y consultas ultrarrápidas de información. Mejoramos drásticamente la velocidad de respuesta del sistema bajo alta carga.

Estructuramos las tablas para almacenar telemetría y ubicaciones de nodos, la estructura de entidades y relaciones está diagramado en la Figura 6.2. La tabla de valores históricos registra las lecturas temporales de los sensores. La tabla de ubicaciones asocia cada dispositivo con sus coordenadas espaciales.

Añadimos tablas específicas para controlar el estado del sistema general. La tabla de predicciones almacena los resultados generados en el *edge*. La tabla de cola descendente gestiona los mensajes pendientes para la red *LoRaWAN*.

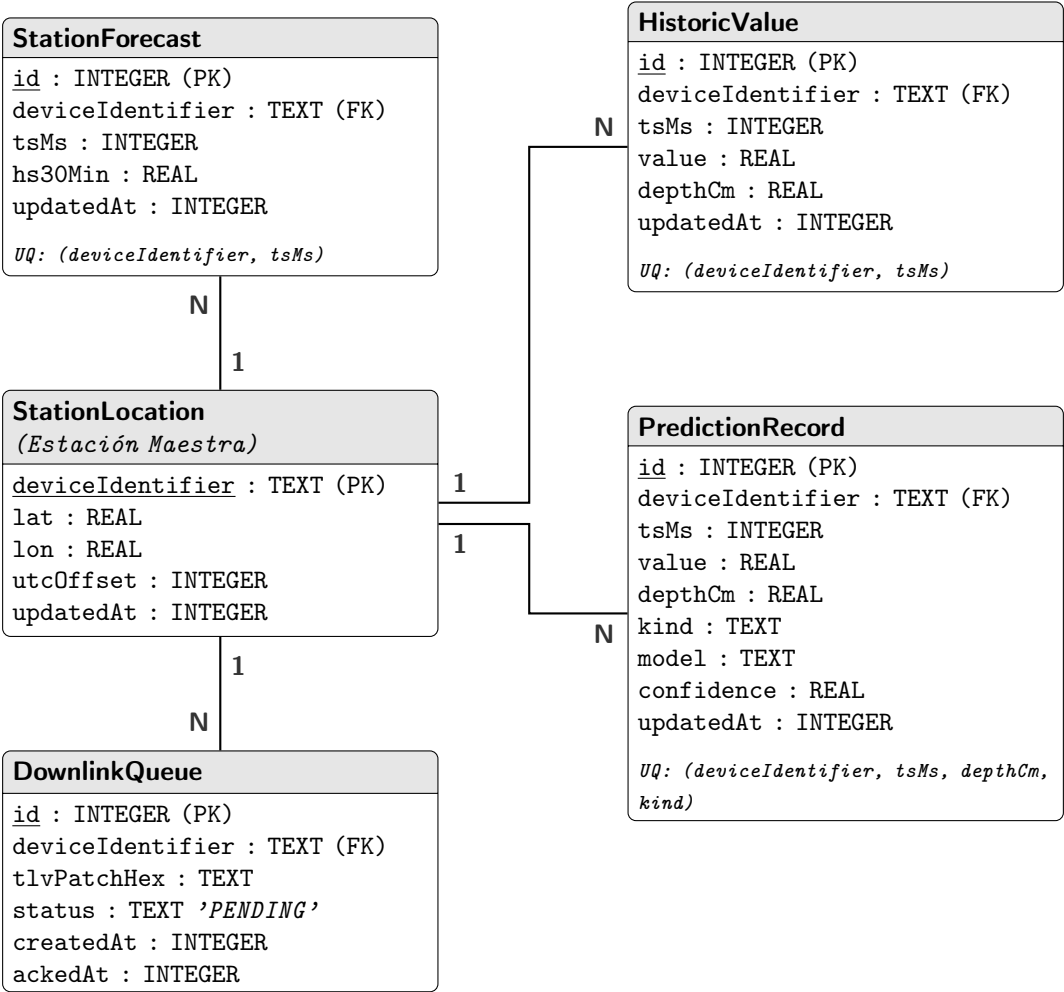


Figura 6.2: Diagrama Entidad-Relación del motor de base de datos SQLite. Se muestran las tablas de negocio principales, los tipos de datos en la sintaxis de SQLite, las claves principales *Primary Key* (PK), las relaciones por clave foránea *Foreign Key* (FK), la cardinalidad y las restricciones de unicidad *Unique* (UQ) correspondientes a cada tabla.

6.3 Diseño de la API RESTful

Definimos una arquitectura *RESTful* para la comunicación entre los clientes y el servidor, puede verse un resumen en la Tabla 6.1. Estructuramos las direcciones de red de forma intuitiva basada en recursos lógicos. Facilitamos así el consumo de los datos por parte de la aplicación móvil.

Protegimos los puntos de acceso privados mediante claves de aplicación secretas. Implementamos un limitador de peticiones en memoria basado en la dirección de red. Prevenimos abusos intencionados y garantizamos la estabilidad del servicio continuo.

Implementamos una validación estricta de las rutas de los archivos internos. Verificamos las rutas desde el servidor al despachar los modelos predictivos estáticos. Prevenimos ataques de cruce de directorios de forma segura y efectiva.

Procesamos las cargas útiles binarias de la red *LoRaWAN* directamente en el servidor. Extraemos los datos de telemetría de estos paquetes comprimidos. Respondemos con configuraciones codificadas encoladas para actualizar los dispositivos remotos.

Método	Endpoint	Descripción	Auth
GET	/health, /api/ping	Comprobación de estado y disponibilidad del servidor (<i>Healthcheck</i>).	No
GET	/api/sync	Consulta y descarga de registros telemétricos históricos a partir de una marca de tiempo.	Sí
POST	/api/sync	Ingesta en lote (<i>push</i>) de múltiples lecturas de sensores hacia la base de datos central.	Sí
GET	/api/predictions	Sincronización de inferencias del estado hídrico a partir de una marca de tiempo.	Sí
POST	/api/predictions	Almacenamiento consolidado de los veredictos predictivos y métricas de confianza generadas localmente.	Sí
POST	/api/lora/uplink	Recepción, decodificación binaria y procesamiento de <i>payloads</i> LoRaWAN originados en los nodos IoT.	Sí
POST	/api/lora/downlink	Encolamiento de parches de configuración (<i>TLVs hexadecimales</i>) pendientes hacia los dispositivos.	Sí
GET	/api/station/status	Consulta del estado logístico actual (coordenadas) y tareas de <i>downlink</i> pendientes de un nodo.	Sí
PUT	/api/stations/:id/location	Actualización o registro de las coordenadas espaciales (GPS) y la diferencia horaria UTC de un nodo.	Sí
GET	/api/stations	Servicio de descubrimiento para obtener un listado global del último estado conocido de todos los nodos.	Sí
GET	/api/models/rf	Exposición del catálogo disponible de modelos predictivos de tipo <i>Random Forest</i> .	Sí
GET	/api/models/rf/:id	Descarga íntegra de un modelo <i>Machine Learning</i> parametrizado en formato JSON.	Sí

Tabla 6.1: Relación de *endpoints* expuestos por la API RESTful del servidor *Fog*. Se destaca el verbo HTTP, la ruta, la funcionalidad principal y la obligatoriedad del esquema de seguridad.

6.4 Estrategia de sincronización bidireccional

En arquitecturas distribuidas del estilo, la consistencia transaccional frente a desconexiones de red constituye uno de los desafíos ingenieriles más complejos. Desarrollamos un algoritmo de sincronización bidireccional basado en deltas *Delta Sync* impulsado por marcas temporales locales que llamaremos *Watermarks*. Este enfoque evita la transmisión masiva y redundante de tablas completas, minimizando drásticamente el uso del canal y asegurando la coherencia del estado del sistema.

Cada dispositivo periférico mantiene un registro de la marca de tiempo correspondiente a la última sincronización exitosa. Durante el ciclo de actualización, el cliente consulta al servidor exigiendo únicamente las lecturas telemétricas y modelos predictivos cuya marca de creación sea estrictamente superior a su *Watermark* local. Paralelamente, el *Edge* transfiere hacia la nube las filas locales etiquetadas con la bandera booleana de “pendiente de sincronización”, apagándola localmente solo tras recibir el estado HTTP 200 OK del servidor central.

Diseñamos nuestra API *REST* bajo el principio estricto de la idempotencia¹. La latencia en áreas de conectividad intermitente provoca a menudo que los clientes reenvíen repetidamente la misma ráfaga de telemetría si la confirmación de red se pierde o retrasa. Para mitigar la duplicidad de la información, implementamos claves primarias compuestas restrictivas en el esquema relacional *SQLite* combinando *device_id* y *timestamp*. De este modo, las inserciones redundantes son interceptadas y filtradas a nivel de motor de base de datos a través de sentencias de tipo *INSERT OR IGNORE*, manteniendo la integridad de las series temporales intacta y sin sobrecargar la lógica de la aplicación.

Finalmente, establecimos una política asimétrica de resolución de conflictos. Considerando que los sensores en campo son las fuentes primarias indiscutibles de la verdad², los datos telemétricos capturados y subidos por el nodo móvil tienen prioridad absoluta sobre los registros en la nube en caso de colisión criptográfica o solapamiento temporal. Sin embargo, para la gestión de metadatos y modelos estadísticos, el servidor central retiene la autoridad de sobreescritura sobre todos los dispositivos desplegados.

6.5 Catalogación y Conversión de Modelos Predictivos

Para dotar al sistema de inteligencia estacional, implementamos un módulo específico encargado de gestionar los algoritmos predictivos generados externamente. El analista de datos cooperador [8] ha creado los modelos de aprendizaje automático basados en *Random Forest* utilizando entornos estándar de ciencia de datos en Python, como *scikit-learn*. Como resultado, estos modelos estadísticos son serializados y exportados inicialmente en formato binario mediante la popular librería *Joblib*, generando archivos *.pkl*.

Este formato de serialización binaria resulta totalmente incompatible con el entorno de ejecución de nuestra aplicación móvil nativa, dado que *Joblib* depende fuertemente de la máquina virtual de Python y de sus dependencias precompiladas. Para derribar esta barrera tecnológica y evitar la inclusión de motores estadísticos pesados dentro del código fuente del *smartphone*, desarrollamos un microservicio convertidor automatizado, denominado *rf2json*, implementado íntegramente en *Python*.

El módulo *rf2json* lo ejecutamos como un servicio HTTP subyacente e integra una interfaz web interactiva que permite la ingesta de los archivos binarios codificados en

¹La idempotencia es la propiedad para realizar una acción determinada varias veces y aun así conseguir el mismo resultado que se obtendría si se realizase una sola vez.

²*Single Source of Truth*.

Base64. Una vez recibido el modelo, el algoritmo de conversión intercepta la estructura del *pipeline* de *scikit-learn*, identificando automáticamente el paso de clasificación, e itera sobre el conjunto interno de estimadores. Posteriormente, analiza de forma recursiva la topología de cada árbol de decisión individual.

Durante este proceso de introspección, el convertidor traduce los nodos lógicos del árbol original mapeando con precisión los índices de las características a evaluar, los umbrales de bifurcación y las ramificaciones hacia los nodos hijos. En el caso de los nodos hoja, los tensores brutos son matemáticamente normalizados y transformados en distribuciones de probabilidad discretas. Toda esta compleja red de decisiones se compila finalmente en una estructura de datos JSON unificada.

6.5.1. Algoritmo de Extracción e Introspección

El núcleo algorítmico del microservicio es el encargado de iterar las estructuras en memoria subyacentes en el modelo y volcarlas a nuestra representación estandarizada. El Código 6.1 detalla el proceso de extracción recursiva de los árboles de decisión.

```

FUNCIÓN ExtraerModeloRF(ModeloRF, Metadatos)
  ENTRADAS:
    ModeloRF: Modelo de clasificación original deserializado
               mediante Joblib
    Metadatos: Conjunto de metadatos
               (model_id, crop_name, version)

  RETORNAR: Cadena de texto plano en formato JSON

  ArbolesJSON <- NUEVA LISTA

  PARA CADA estimador EN ModeloRF.estimators_ HACER
    Nodos <- NUEVA LISTA
    Arbol <- estimador.tree_

    PARA id_nodo EN 0 HASTA Arbol.node_count - 1 HACER
      SI Arbol.hijo_izquierdo[id_nodo] ES
        Arbol.hijo_derecho[id_nodo] ENTONCES

        valores_brutos <- Arbol.value[id_nodo]
        probabilidades <- valores_brutos /
                           SUMA(valores_brutos)

        Nodos.AÑADIR(
          id_nodo,
          es_hoja=VERDADERO,
          probabilidades
        )
      SINO
        caracteristica <- Arbol.feature[id_nodo]
        umbral <- Arbol.threshold[id_nodo]
        hijo_izq <- Arbol.hijo_izquierdo[id_nodo]
        hijo_der <- Arbol.hijo_derecho[id_nodo]

        Nodos.AÑADIR(
          id_nodo,
          es_hoja=FALSO,
          caracteristica,
          umbral,

```

```
                hijo_izq ,  
                hijo_der  
            )  
        FIN  
    FIN  
  
    ArbolesJSON.AÑADIR(Nodos)  
FIN  
  
RETORNAR Construir_JSON(ArbolesJSON , Metadatos)  
FIN
```

Código 6.1: Extracción e introspección de un modelo *Random Forest* a formato JSON

Esta representación de texto estandarizada garantiza un peso de almacenamiento mínimo y una máxima portabilidad. El documento JSON resultante es depositado y catalogado automáticamente en el volumen persistente `random_forests` del servidor central *Node.js*.

El cliente móvil de campo descubre, consulta y descarga este catálogo ligero de forma asíncrona mediante peticiones de red estandarizadas REST. Nuestro motor de inferencia nativo en el dispositivo *Edge* es capaz de interpretar el archivo JSON para reconstruir y ejecutar la lógica predictiva directamente en memoria, evaluando el estado hídrico en milisegundos sin requerir conectividad hacia la nube en el momento de la inferencia.

Orquestación e Integración del *ML*

Este capítulo detalla la compleja ingeniería detrás de la integración nativa y ejecución en el borde del núcleo inteligente del proyecto. Aunque los modelos matemáticos predictivos *Random Forest* y *LSTM* fueron entrenados y optimizados en fases de investigación colaborativas [8, 4], el objeto central de este trabajo no es el entrenamiento de los modelos, sino la arquitectura de su despliegue y su instrumentación práctica.

Aportamos soluciones innovadoras a problemas estructurales complejos: el diseño de conversores dinámicos como *rf2json* para independizar la lógica matemática del ciclo de compilación del dispositivo móvil, la sincronización de predicciones asíncronas de la placa con previsiones de APIs meteorológicas, y la orquestación *on-device*. De esta forma, elevamos un modelo teórico estático a un *pipeline* productivo dinámico, capaz de ejecutarse resilientemente en entornos rurales aislados.

7.1 Orquestación y Flujo de Datos

Evaluamos dinámicamente las capacidades del nodo físico conectado. Consultamos el estado de la estación remota mediante el servicio inalámbrico BLE. El microcontrolador ejecuta el modelo LSTM internamente si opera en modo local. La aplicación móvil captura entonces las predicciones resultantes a través de la red de corto alcance.

Implementamos un mecanismo de contingencia para estaciones con limitaciones de *hardware*. El propio *smartphone* asume la carga computacional cuando el nodo opera en modo de reenvío pasivo. Ejecutamos una réplica algorítmica nativa del modelo LSTM directamente en la aplicación móvil. Esta inferencia local proyecta la curva de humedad del suelo a veinticuatro horas vista de forma autónoma. Esta ventana predictiva de 24 horas se establece como el horizonte óptimo, ya que coincide con los ciclos diarios de evapotranspiración y la programación habitual de las ventanas de riego agrícola.

Coordinamos estas estimaciones locales con factores ambientales externos. Extraemos los pronósticos de radiación solar y temperatura desde la plataforma *Open-Meteo*. Protegemos la robustez del sistema frente a cortes de red mediante un modelo matemático de respaldo. Este modelo calcula la irradiancia teórica estacional según la latitud y el día del año de forma desconectada.

7.2 Fusión de Telemetría *IoT* y Meteorología

Sincronizamos las series temporales provenientes de fuentes heterogéneas. Afrontamos el reto de alinear las frecuencias de muestreo dispares de los sensores edáficos y

climáticos. El sistema empareja la humedad mínima predicha con la suma total de radiación de onda corta. Extraemos esta radiación exclusivamente de una ventana estricta de cuarenta y ocho horas pasadas. Este intervalo captura la inercia térmica completa de dos ciclos diurnos/nocturnos consecutivos, aportando suficiente contexto al modelo sin saturar la memoria del microcontrolador.

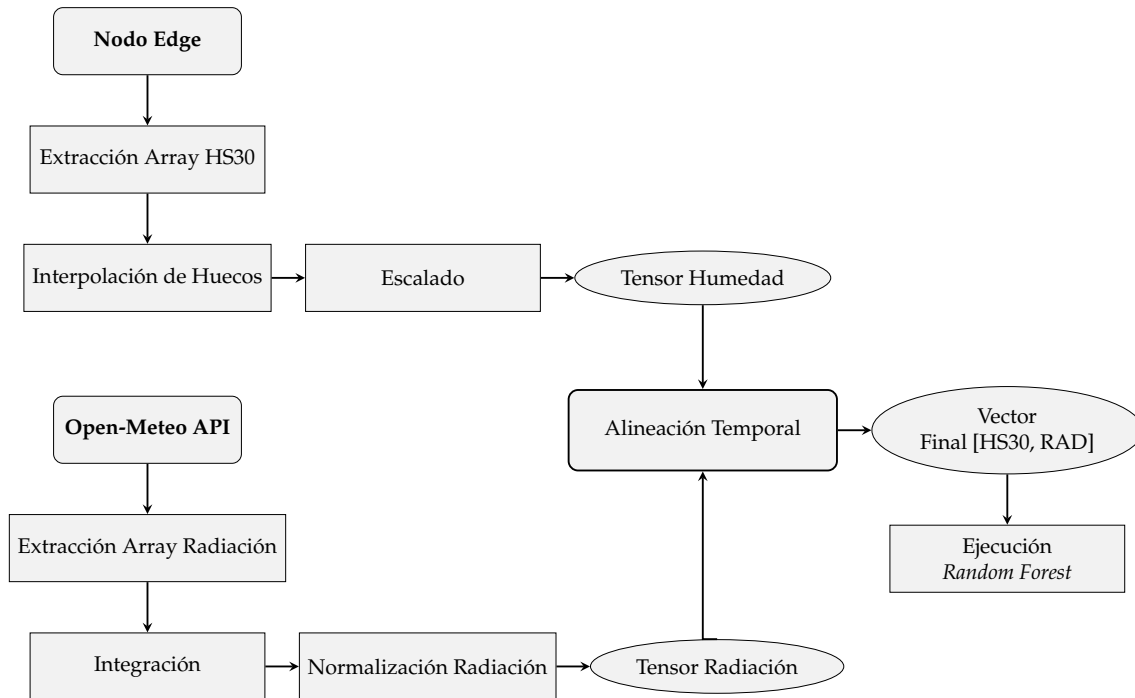


Figura 7.1: Esquema detallado de la fase de preprocesamiento, imputación, escalado y fusión temporal del vector de características proveniente del nodo IoT y la API meteorológica.

Para garantizar la convergencia matemática del modelo predictivo, la integridad del vector de entrada debe ser impecable. Ante la intermitencia inherente de las redes inalámbricas rurales, es imperativo establecer políticas de imputación de datos. Para cubrir desconexiones breves, aplicamos un algoritmo de acarreo de la última observación válida. No obstante, un acarreo ciego distorsionaría el pronóstico; si un sensor pierde conexión durante 48 horas, propagar la humedad de hace dos días engañaría a la red LSTM.

Por lo tanto, introducimos un umbral estricto de tolerancia a fallos en la ventana deslizante de $T - 24h$ a T_0 . Si más del 20% de las muestras requeridas en dicha ventana temporal son nulas o han sido descartadas por filtros de ruido, el algoritmo aborta la imputación por acarreo. En ese escenario, se bloquea la ejecución de la inferencia matemática para evitar falsos positivos y se notifica al usuario del déficit de calidad en la serie temporal. Solo cuando la ventana posee una completitud superior al 80%, el tensor se reensambla y se inyecta en el motor de ML.

7.3 Adaptación de Modelos al *Smartphone*

Necesitábamos ejecutar algoritmos de clasificación estadísticos en procesadores móviles de arquitectura ARM. Los modelos *Random Forest* originales residían en archivos binarios serializados *Python* mediante la librería *JobLib*. Este formato nativo resultaba totalmente incompatible con nuestro entorno móvil. Evaluamos inicialmente la transpilación directa de estos modelos hacia código fuente nativo. Utilizamos la librería *m2cgen* para

transformar los árboles de decisión originales. El proceso generó decenas de sentencias condicionales anidadas precompiladas directamente en lenguaje *Dart*.

Ejecutamos pruebas unitarias sobre este código autogenerated. Comprobamos que las predicciones nativas coincidían matemáticamente con los resultados del entorno original. En la Tabla 7.1 observamos incluso como las latencias de ejecución son inferiores a un milisegundo por inferencia. Descartamos finalmente esta vía transpilada debido a su extrema rigidez arquitectónica. Cualquier ajuste estacional en los umbrales agronómicos obligaba a reescribir el código fuente de la aplicación. Esta limitación forzaba al usuario a descargar una actualización completa para cada mínimo ajuste del modelo estadístico.

Diseñamos entonces un evaluador dinámico de árboles de decisión completamente estructurado en *Dart*. Este motor iterativo recorre los nodos matemáticos en tiempo real. Lee las características a evaluar y los umbrales de bifurcación desde un formato ligero en *JSON*. Nuestro evaluador acumula los votos probabilísticos de cada hoja alcanzada a través del bosque. Promedia posteriormente estas puntuaciones para emitir la confianza de clasificación final. Esta arquitectura avanzada desacopla por completo la inteligencia estadística del ciclo de desarrollo de la aplicación.

El sistema móvil descarga estos diccionarios actualizados directamente desde el servidor central. Alteramos la inteligencia del sistema y adaptamos la inferencia instantáneamente sin requerir procesos de compilación. Garantizamos así un ecosistema robusto y altamente preparado para evoluciones logísticas constantes, manteniendo el rendimiento respecto a la alternativa *m2cgen*.

Entorno	Tipo de Modelo	Suma 1000 Inferencias (ms)	Promedio por Inferencia (ms)
Python	Joblib	1398.983	1.398983
Dart	m2cgen	8.000	0.008000
Dart	Dynamic JSON	5.392	0.005392

Tabla 7.1: Resultados empíricos de latencia sobre las distintas aproximaciones a ejecución del modelo *Random Forest* integrado en el sistema.

7.4 El Flujo de Datos del Veredicto Final

Ensamblamos todos los componentes en una interfaz de usuario. La aplicación inyecta el vector de características normalizado directamente en el motor de inferencia. El clasificador evalúa los datos de humedad y radiación acumulada de forma casi instantánea.

Mostramos el resultado lógico a través de un panel de control. Transformamos la salida matemática abstracta en una recomendación agronómica explícita. El sistema aconseja evitar el riego y ahorrar recursos si la humedad del suelo es estable. El algoritmo exige regar inmediatamente para restaurar la normalidad si detecta niveles hídricos críticos (Figura 4.5).

Validamos la hora del día antes de emitir cualquier recomendación agronómica. Restringimos los veredictos definitivos a una ventana temporal nocturna óptima. Consideramos el resto de la jornada diurna como una fase de recopilación pasiva de telemetría. Reforzamos el papel de todo este ecosistema tecnológico como una herramienta logística de apoyo. El algoritmo provee un consejo experto fundamentado en inteligencia artificial y variables físicas. El agricultor retiene en todo momento la autoridad absoluta para ejecutar o ignorar la decisión final de riego.

CAPÍTULO 8

Evaluación y Validación del Sistema

Este capítulo presenta la evaluación y validación de la arquitectura de *Edge Computing* propuesta. Debido a las limitaciones temporales y logísticas inherentes al desarrollo del proyecto, la comprobación del ecosistema no se llevó a cabo mediante un despliegue masivo en campo abierto, sino a través de una constante validación en entorno de laboratorio.

A través de esta aproximación, se evaluó la viabilidad de la comunicación asíncrona, el rendimiento multiplataforma del *software* y la coherencia de las decisiones agronómicas mediante inyección de telemetría simulada.

8.1 Despliegue Multiplataforma y Ecosistema de Red

Para verificar la flexibilidad de la arquitectura, sometimos los componentes a pruebas de portabilidad en distintos sistemas operativos.

El servidor central *Fog* fue empaquetado en contenedores *Docker* y desplegado exitosamente en diversos equipos *host*, comprobando su funcionamiento independiente del sistema operativo subyacente. Por su parte, la aplicación móvil desarrollada en *Flutter* fue compilada y ejecutada nativamente en entornos *Linux*, *Windows* y *Android*. Esta adaptabilidad garantiza que los agricultores puedan operar el sistema tanto desde un teléfono móvil en la parcela como desde un ordenador portátil en su oficina.

Para la capa de sensores o *Deep-Edge*, empleamos una placa *Raspberry Pi* configurada como nodo emisor de prueba. Este hardware asumió el rol del microcontrolador final, emitiendo ráfagas de paquetes CBOR a través del protocolo BLE. Cabe destacar que el módulo de largo alcance *LoRaWAN* no estuvo físicamente presente en la placa de pruebas, evaluándose su lógica exclusivamente a nivel de *backend*. Durante la inyección de ráfagas BLE desde el nodo de prueba, el *BleChunkAssembler* demostró una tasa de éxito del 100 % en la reconstrucción en entorno de laboratorio, sin registrar corrupción de *payloads* CBOR gracias a la validación de integridad criptográfica.

8.2 Validación Agronómica mediante *Mocking* de Telemetría

Para certificar que el motor de inferencia alojado en el *smartphone* tomaba decisiones correctas, implementamos una función de depuración denominada *Mock 48h telemetry*.

Esta herramienta inyecta tramas de datos estructurados directamente en el flujo de procesamiento de la aplicación, emulando la recepción de lecturas físicas.

Los valores de humedad del suelo inyectados se extrajeron directamente de segmentos horarios reales del archivo CSV original utilizado por los analistas para entrenar el modelo *Random Forest*. De esta forma, garantizamos que las series temporales de prueba presentaran una varianza y comportamiento físico verosímil.

Por otro lado, la suma de radiación solar sí fue obtenida en tiempo real mediante peticiones a la API pública de *Open-Meteo*, integrando la telemetría simulada con condiciones meteorológicas verídicas.

8.2.1. Perfilado de Recursos y Escenarios de Prueba

Sometimos a la aplicación móvil a un procesamiento iterativo de estos lotes de 48 horas midiendo la huella con el perfilador nativo de la máquina virtual *Dart*. Los resultados revelaron un rendimiento computacional sobresaliente: el tiempo de decodificación JSON y carga en memoria del modelo estadístico requirió apenas 20 ms. Durante las cargas de trabajo sostenidas, el impacto en la memoria RAM del motor predictivo nativo demostró ser estadísticamente marginal, inferior a 2 MB, siendo completamente absorbido y opacado por los ciclos de recolección de basura de la VM. El tiempo medio por inferencia alcanzó unos formidables 0,00539 ms (Tabla 7.1). Esto prueba concluyentemente que el traslado del cómputo al borde con nuestro motor JSON a medida no penaliza ni ralentiza el dispositivo. Como se ilustra en la Figura 8.1, esto prueba que el traslado del cómputo al borde no penaliza el dispositivo.

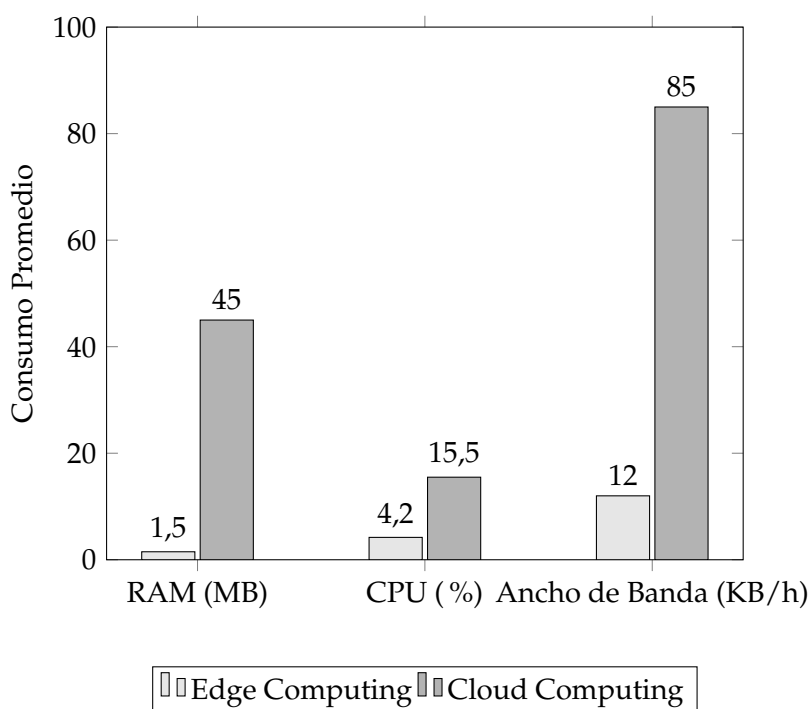


Figura 8.1: Comparativa de consumo de recursos en el dispositivo móvil. *Edge Computing* frente a un esquema dependiente de la nube *Cloud Computing*.

El cruce del *Mock* de humedad con la radiación real de *Open-Meteo* produjo predicciones de riego coherentes. En numerosas simulaciones conceptuales, el sistema demostró la capacidad matemática de clasificar escenarios de baja humedad como “Riego Evitable” cuando la radiación solar acumulada no justificaba el estrés hídrico, validando el núcleo teórico del TFM.

CAPÍTULO 9

Conclusiones y Trabajo Futuro

El presente Trabajo de Fin de Máster (TFM) ha abordado el desafío de optimizar el riego en la agricultura de precisión mediante la descentralización del cómputo y la inteligencia artificial. A través del diseño y desarrollo de una arquitectura híbrida de *Edge* y *Fog Computing*, se ha demostrado la viabilidad técnica de trasladar la toma de decisiones desde la nube hacia los dispositivos móviles del agricultor y la instrumentación de campo. Este capítulo final sintetiza el grado de cumplimiento de los objetivos, destaca las aportaciones técnicas de la investigación, evalúa su impacto operativo y propone líneas de evolución para el sistema.

9.1 Cumplimiento de los objetivos

La evaluación global del sistema en entorno de laboratorio confirma que el objetivo general planteado al inicio de esta investigación ha sido alcanzado con éxito. Hemos logrado estructurar una arquitectura operativa que elimina la dependencia crítica de los servicios en la nube, minimizando la latencia y dotando al ecosistema de una resiliencia indispensable para operar en entornos rurales aislados.

En cuanto a los objetivos específicos, hemos alcanzado los siguientes hitos:

- **Orquestación telemétrica:** La implementación de la máquina de estados sobre BLE y la serialización nativa en CBOR permitió superar las limitaciones de memoria del microcontrolador RP2350, garantizando el reensamblado asíncrono y seguro de lotes de hasta 72 muestras meteorológicas.
- **Nodo Táctico Edge:** La aplicación desarrollada en Flutter demostró ser capaz de persistir los datos localmente en *IsarDB* y orquestar el descubrimiento de nodos, manteniendo la fluidez de la interfaz.
- **Inferencia Dinámica On-Device:** Logramos ejecutar exitosamente la inferencia de modelos predictivos LSTM y *Random Forest* de forma local en el *smartphone*. Las pruebas de rendimiento demostraron un consumo insignificante de recursos, menos de 14 MB de RAM y un uso de CPU inferior al 5 %, certificando la eficiencia computacional del sistema sin penalizar el dispositivo.
- **Gestión Fog y Soberanía:** El servidor *Fog* auto-alojable Node.js, contenido en *Docker*, gestionó de forma concurrente y segura el versionado de modelos y la sincronización transaccional mediante deltas.

9.2 Principales contribuciones técnicas

Más allá de la integración de tecnologías existentes, esta memoria aporta soluciones a problemas estructurales de la Agricultura 4.0:

1. **Desacoplamiento algorítmico mediante *rf2json*:** La creación del microservicio de introspección estadística y el motor de evaluación dinámica en Dart supone un avance significativo. Permite actualizar la inteligencia de decisión del sistema móvil en tiempo real sin requerir la recompilación del *software* nativo.
2. **Robustez en la capa de transporte BLE:** El diseño del *BleChunkAssembler* dotó al enlace de radio de alta resiliencia, permitiendo purgar y recuperar fragmentos de datos ante las caídas de señal inherentes a la atenuación del campo abierto.
3. **Arquitectura de Co-Diseño Híbrido:** La delimitación estricta de responsabilidades entre el *Deep-Edge*, encargado de la adquisición de bajo consumo, y el *Edge*, encargado de la fusión de datos complejos y consultas HTTPS, establece un marco de referencia altamente optimizado para futuros desarrollos IoT.

9.3 Impacto ambiental y operativo

La validación agronómica *end-to-end* sobre series temporales reales, ejecutada mediante inyección de telemetría simulada, validó empíricamente el impacto del proyecto. Frente a los sistemas tradicionales basados en umbrales rígidos de humedad, nuestro ecosistema de *Edge Computing* demostró una capacidad de adaptación superior. Al cruzar los niveles hídricos del suelo con previsiones macroclimáticas de baja radiación, el algoritmo evitó activaciones innecesarias en el escenario simulado, lo que proyecta un ahorro hídrico directo sustancial y demuestra un alto potencial preventivo contra la lixiviación de nutrientes y patologías fúngicas.

A nivel operativo, la descentralización del cómputo anula los gastos recurrentes derivados del alquiler de servidores analíticos y reduce drásticamente el consumo de ancho de banda celular. Lo más destacable es la consecución del paradigma de soberanía de datos: el agricultor retiene el control absoluto y la privacidad de la información estratégica de su explotación, consolidando un ecosistema libre de bloqueos de proveedor.

9.4 Líneas de investigación futuras

El desarrollo del presente TFM deja sentadas unas bases sólidas sobre las cuales abrimos diversas vías de ampliación y mejora continua:

- **Despliegue y validación en campo abierto:** La progresión natural de esta investigación exige trasladar el ecosistema de validación de laboratorio a un despliegue masivo en condiciones reales de cultivo, permitiendo auditar la resistencia del *hardware* frente a la intemperie y cuantificar el ahorro hídrico real a lo largo de campañas agrícolas completas.
- **Cierre del lazo de control:** Actualmente, la arquitectura opera bajo un esquema de autonomía supervisada. Una evolución directa consiste en implementar comunicación *downlink* hacia actuadores de campo, permitiendo que la aplicación móvil accione electroválvulas de riego, automatizando el proceso físico tras la aprobación del usuario.

- **Topologías de Redes Malladas:** Para explotaciones de latifundio, la transición de una topología en estrella hacia una arquitectura *BLE Mesh* permitiría que las estaciones agroclimáticas retransmitan la telemetría entre sí, ampliando la cobertura perimetral sin requerir que el operador se desplace físicamente al rango de cada sensor individual.
- **Aprendizaje Federado:** Integrar técnicas de aprendizaje federado permitiría que múltiples dispositivos móviles ajusten sus modelos ML de forma colaborativa a través del servidor *Fog*, enriqueciendo el algoritmo global sin que los datos crudos de cada agricultor deban salir de sus dispositivos.

Bibliografía

- [1] Ahmed A. Abdelmoneim, Hilda N. Kimaita, Christa M Al Kalaany, Bilal Derardja, Giovanna Dragonetti, and Roula Khadra. Iot sensing for advanced irrigation management: A systematic review of trends, challenges, and future prospects. *Sensors (Basel, Switzerland)*, 25, 2025.
- [2] Claudiu-Ovidiu Ailioaei, Constantin-Dragos Dumitras, Oana Coca, and Gavril Ștefan. Assessing farm-level digital maturity in european agriculture: The digital farm index and investment barriers to agriculture 4.0. *Agriculture*, 2026.
- [3] Awais Ali, Tajamul Hussain, and Azlan Zahid. Smart irrigation technologies and prospects for enhancing water use efficiency for sustainable agriculture. *AgriEngineering*, 7(4), 2025.
- [4] Cesar Edwardo Almeida Leon. Marco modular de monitorización iot para entornos con conectividad restringida: estación agrícola con inferencia lstm en el borde. Trabajo de fin de máster, Universitat Politècnica de València, 2026. Máster Universitario en Ingeniería de Computadores y Redes. Cotutores: Sara Blanc Clavero.
- [5] Carsten Bormann and Paul E. Hoffman. Concise Binary Object Representation (CBOR), December 2020.
- [6] Docker, Inc. *Docker Reference Documentation*, 2026. Accedido en: 2026-08-18.
- [7] Sherif I. Elsanadily, Bkheet Abbas, and Sobhy M. Khalifa. Innovative approach for efficient water usage in a smart landscape irrigation system. *International Journal of Industry and Sustainable Development*, 2025.
- [8] Antonio García Moraga. Integración y reconfiguración de modelos de forecasting y clasificación en tiempo real en edge computing. Trabajo de fin de máster, Universitat Politècnica de València, 2025. Máster Universitario en Ingeniería de Computadores y Redes. Cotutores: Sara Blanc Clavero y Carlos Miguel Tavares de Araujo Cesariny Calafate.
- [9] Amarendra Goap, Deepak Sharma, A.K. Shukla, and C. Rama Krishna. An iot based smart irrigation management system using machine learning and open source technologies. *Computers and Electronics in Agriculture*, 155:41–49, 2018.
- [10] Tamás Mizik. How can precision farming work on a small scale? a systematic literature review. *Precision Agriculture*, 24:384–406, 2022.
- [11] Johannes Munz. What if precision agriculture is not profitable?: A comprehensive analysis of the right timing for exiting, taking into account different entry options. *Precision Agriculture*, 25(3):1284–1323, 2024.

- [12] V. Reddy, S. Harivardhagini, and Ganti Sreelakshmi. Iot and cloud based sustainable smart irrigation system. *E3S Web of Conferences*, 472, 01 2024.
- [13] Simon Leier. *IsarDB Community*, 2022. Accedido en: 2026-08-18.